AI Agent 개발

가천대 부트캠프

LangChain



Contents

1. What is LangChain
   - Agent
   - 랭체인이란?
   - 랭체인 시작하기

2. Model
   - Model I/O
   - parameter
   - 주요 메서드
   - 메모리
   - 구조화된 답변

3. Agents
   - Tool 기초
   - Tool 응용
   - LangChain 통합 Tools
   - Agent 메모리

4. 참고 자료
   - System Prompt
   - ReAct & Lost in the Middle
   - 랭그래프 & 랭스미스

제 1장.

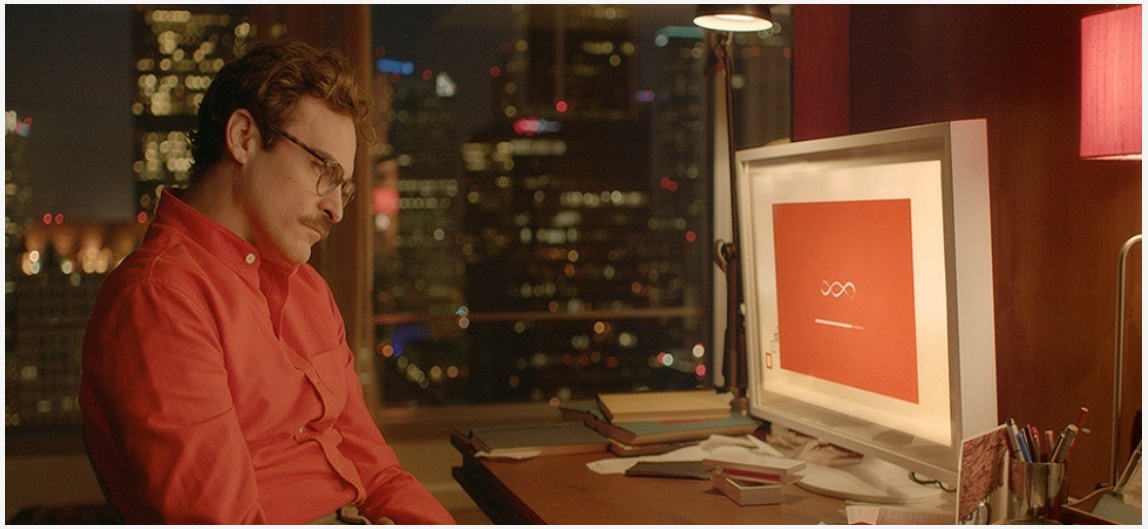
What is LangChain

1-1 Agent

What is Agent?

Agent





Agent

목표를 달성하기 위해 **스스로 판단**하고 도구를 사용해 **행동**하는 시스템

Agent



Agent



〈 생성형 AI 등장 전 〉



〈 ChatGPT 등장 〉

AI 어시스턴트



〈 Codex 등장 〉

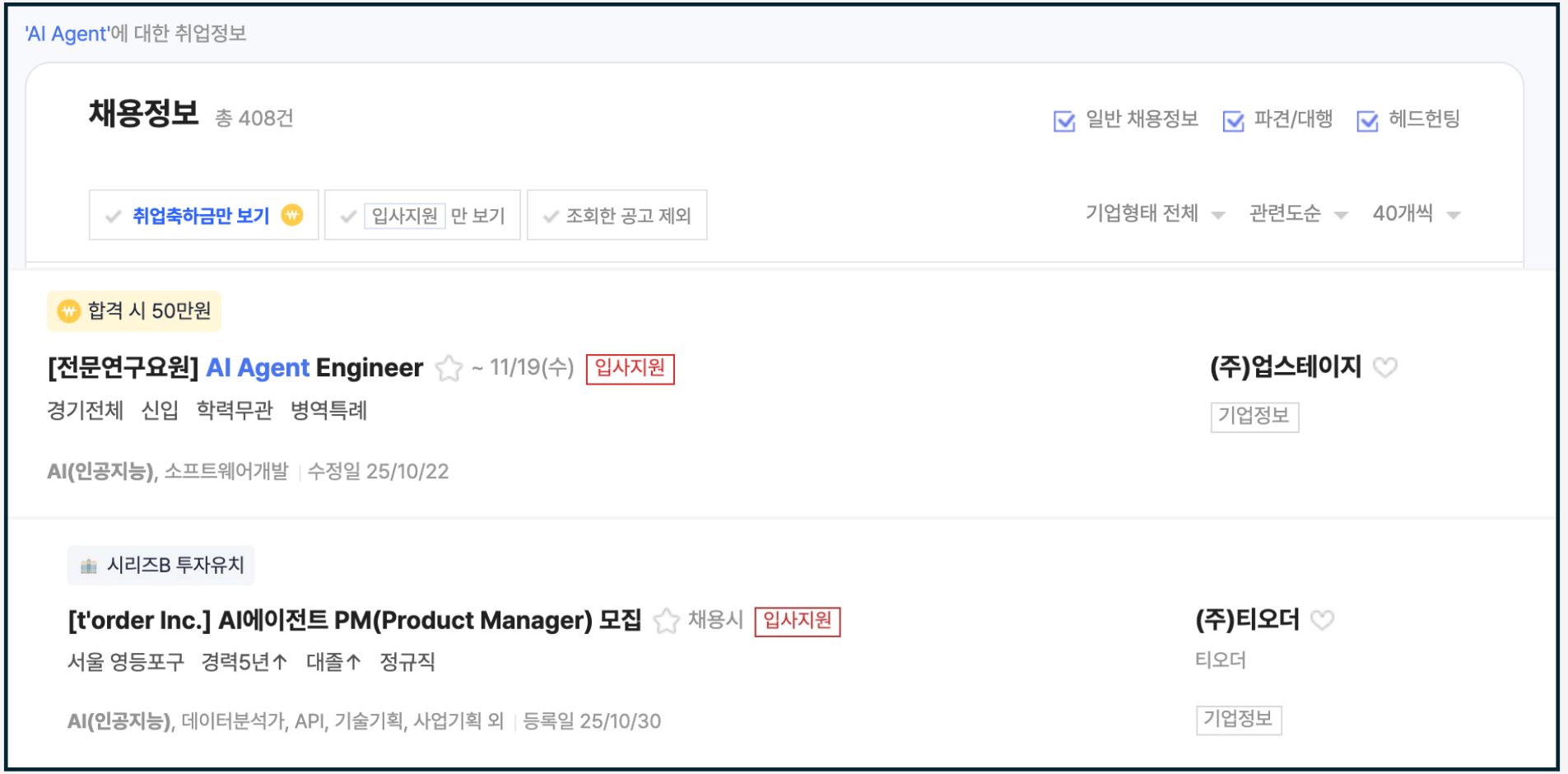
AI 에이전트

Agent

- **기존** : 개발자가 직접 코드 작성
- **코딩 어시스턴트** : 개발자가 AI의 도움을 받아 코드 작성 및 수정
  - 개발자 : 이 기능 어떻게 구현해?
  - AI : 이 기능은 이렇게 코드 작성하면 됩니다.,
  - 개발자 : 코드 검수 후 구현
- **코딩 에이전트** : AI가 코드 작성 및 수정
  - 개발자 (사람) : 이 기능 구현해 줘.
  - AI : 구현 완료됐습니다.

👉 AI가 직접 코드 작성, 인간은 검수

Agent



출처 : https://www.saramin.co.kr/zf_user/search?search_area=main&search_done=y&search_optional_item=n&searchType=search&searchword=AI%20Agent

Agent

LangChain

→ LLM 기반 애플리케이션과 Agent를 빠르고 쉽게 개발할 수 있는

프레임워크 (Framework)

출처 : https://docs.langchain.com/oss/python/langchain/overview

1-2 랭체인이란?

랭체인이란?

LangChain

→ LLM 기반 애플리케이션과 Agent를 빠르고 쉽게 개발할 수 있는

프레임워크 (Framework)

출처 : https://docs.langchain.com/oss/python/langchain/overview

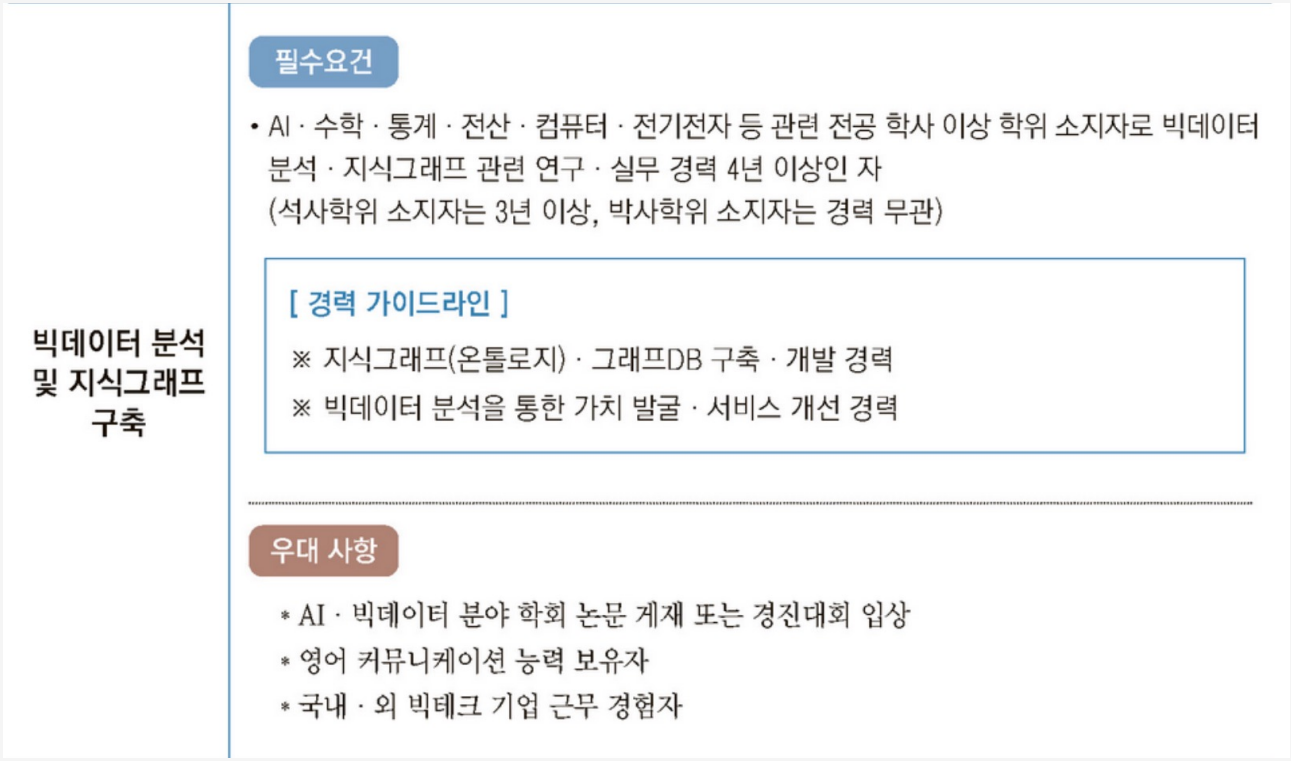
랭체인이란?

- 2026년 국가정보원 채용공고

모집분야	내 용
에이전트 및 MCP 개발 · 운영	필수요건 • AI · 수학 · 통계 · 전산 · 컴퓨터 · 전기전자 등 관련 전공 학사 이상 학위 소지자로 AI 에이전트 시스템 설계 · 운영 등 관련 실무 경력 2년 이상인 자 (석사학위 소지자는 1년 이상, 박사학위 소지자는 경력 무관) [요구 경력] ※ Python, Javascript, Java 등을 활용한 AI 애플리케이션 개발 경력 ※ AI 에이전트 시스템 설계 · 운영 경력 우대 사항 * Langchain, MCP 등 에이전트 기반 기술 활용 경험자 * AI 에이전트 기반 배포 파이프라인 설계 · 운영 능력 보유자 * 국내 · 외 빅테크 기업 근무 경험자

랭체인이란?

- 2026년 국가정보원 채용공고



빅데이터 분석 및 지식그래프 구축

필수요건

• AI · 수학 · 통계 · 전산 · 컴퓨터 · 전기전자 등 관련 전공 학사 이상 학위 소지자로 빅데이터 분석 · 지식그래프 관련 연구 · 실무 경력 4년 이상인 자
(석사학위 소지자는 3년 이상, 박사학위 소지자는 경력 무관)

[경력 가이드라인]

※ 지식그래프(온톨로지) · 그래프DB 구축 · 개발 경력

※ 빅데이터 분석을 통한 가치 발굴 · 서비스 개선 경력

우대 사항

* AI · 빅데이터 분야 학회 논문 게재 또는 경진대회 입상
* 영어 커뮤니케이션 능력 보유자
* 국내 · 외 빅테크 기업 근무 경험자

랭체인이란?

- 랭체인
  - LLM 기반 애플리케이션과 Agent를 빠르고 쉽게 개발할 수 있는 **프레임워크 (Framework)**

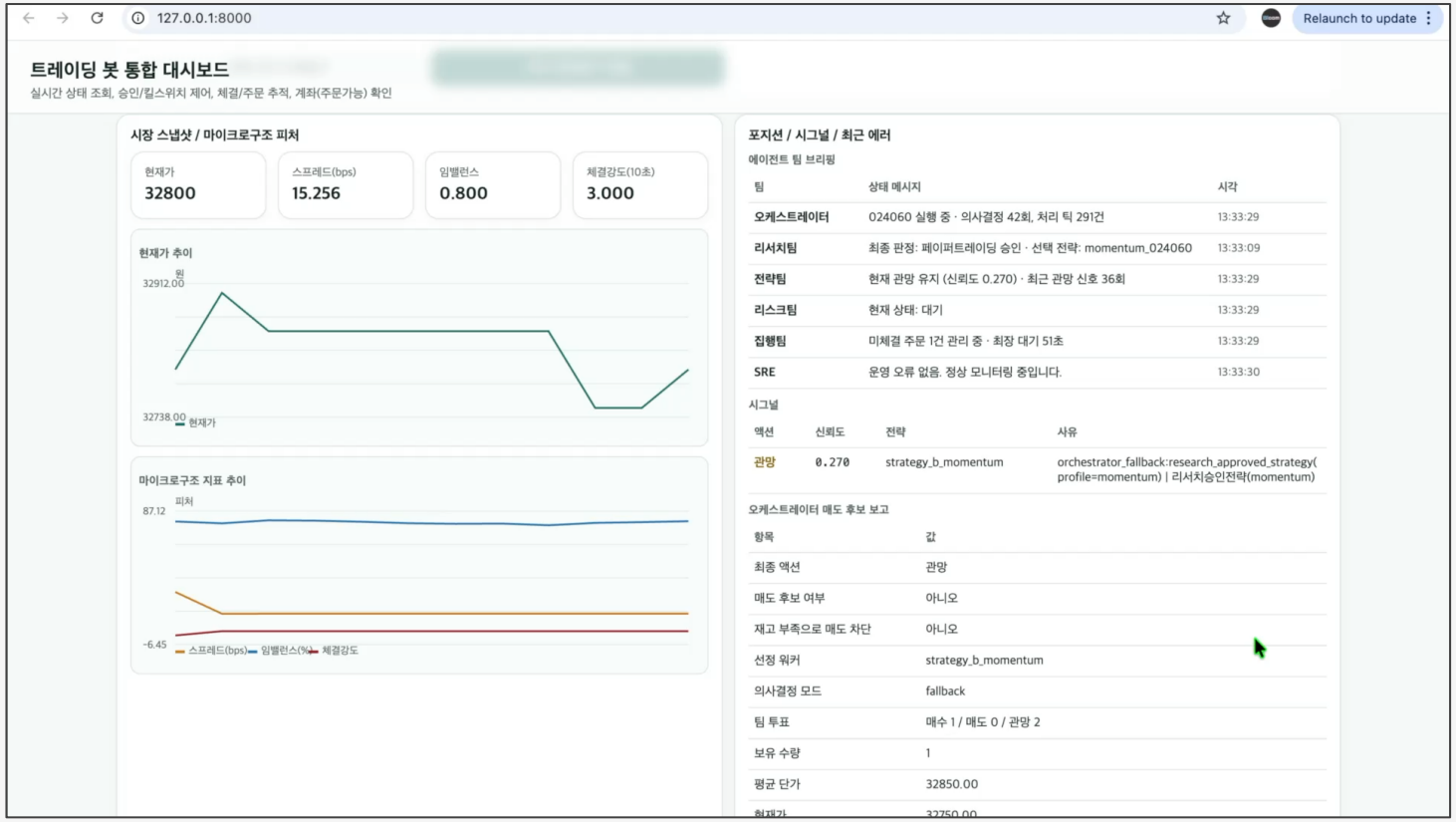
$$BM25 = \sum_{t \in q} \log \left[\frac{N}{df(t)} \right] \cdot \frac{(k_1 + 1) \cdot tf(t,d)}{k_1 \cdot \left[(1-b) + b \cdot \frac{dl(d)}{dl_{avg}} \right] + tf(t,d)}$$

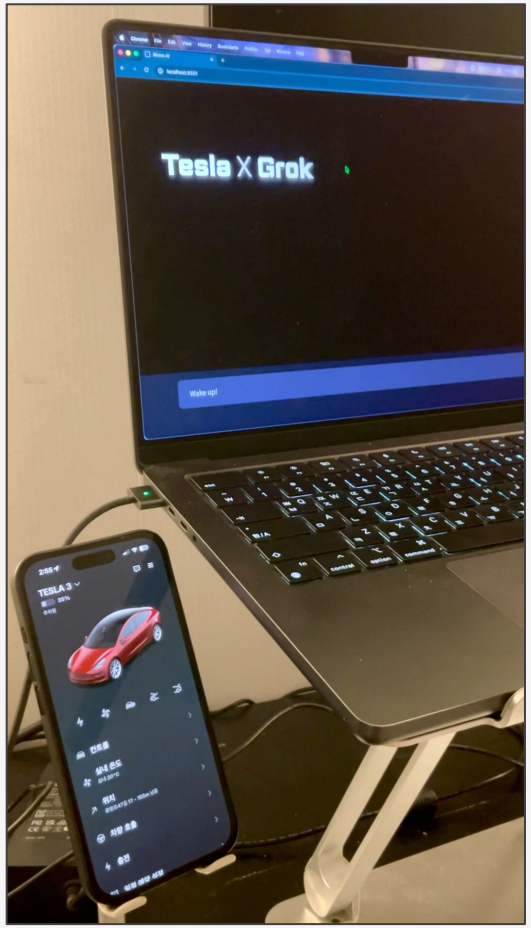
```python
from langchain_community.retrievers import BM25Retriever

retriever = BM25Retriever.from_texts()
```

랭체인이란?

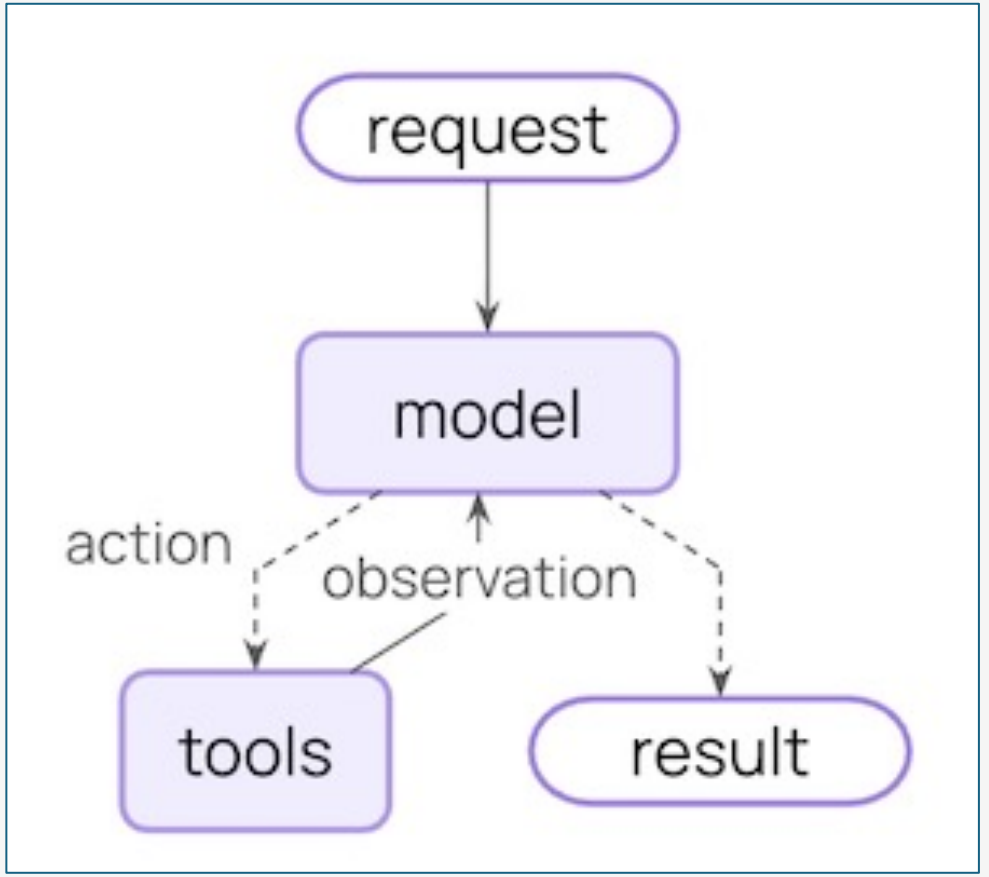
- 랭체인 활용 프로젝트





랭체인이란?

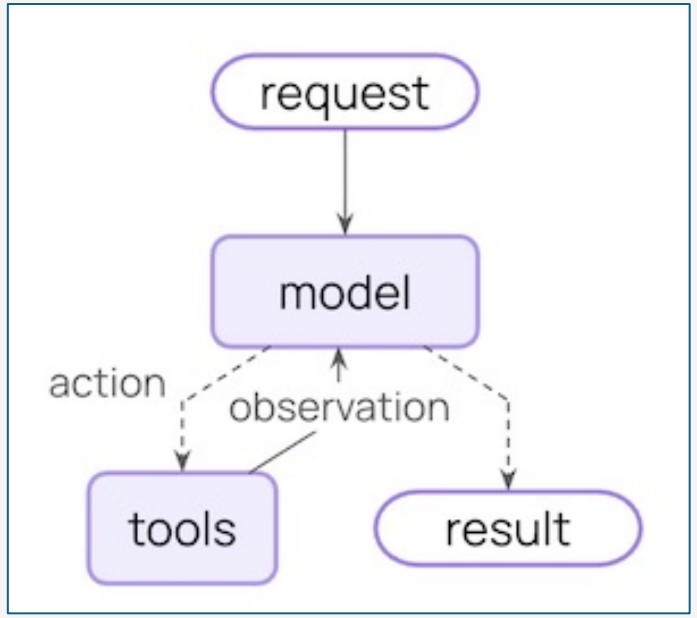
- Agent Architecture
  - 사용자의 입력(질문, 지시)에 대해 적절한 행동을 취하도록 **추론(Reasoning)**한 뒤 **행동(Act)**하는 **주체(Excutor)**
  - 구성 요소
    - Model (LLM)
    - Tools (함수)



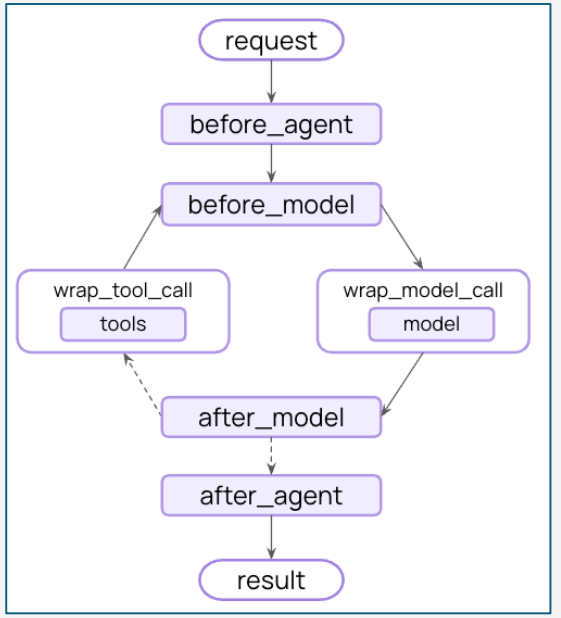
출처 : https://docs.langchain.com/oss/python/langchain/agents

랭체인이란?

- 랭체인 v1
  - 2022년 10월 : V 0.0.1 등장
  - 2025년 10월 : V 1.0.0 공개 → Agent 구축에 집중



〈Basic〉



〈Adv〉

출처 : https://docs.langchain.com/oss/python/releases/langchain-v1

랭체인이란?

- 랭체인 학습 가이드
  - **공식 문서** : https://docs.langchain.com/
  - **공식 블로그** : https://www.langchain.com/blog
  - **공식 아카데미** : https://academy.langchain.com/

랭체인이란?

High level language





High level Framework





Low level language





Low level Framework

1-3 랭체인 시작하기

랭체인 시작하기

- 랭체인 패키지 설치
  - -U 또는 --upgrade : 최신 버전으로 설치

```
pip install -U langchain
```

  - 실행 결과

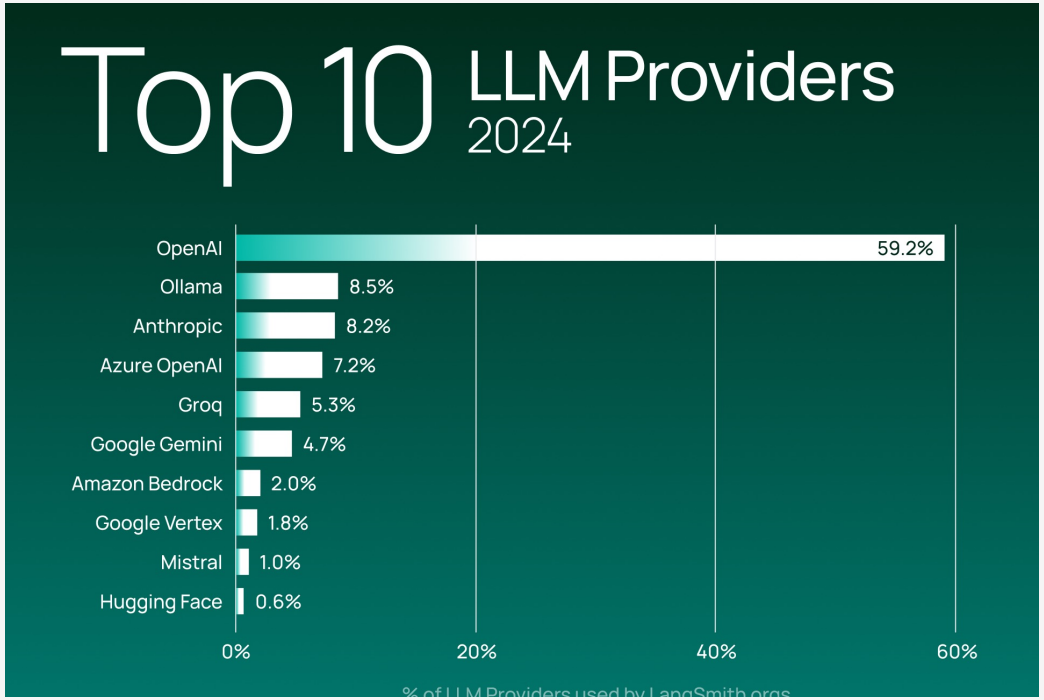
```
!pip install -U langchain

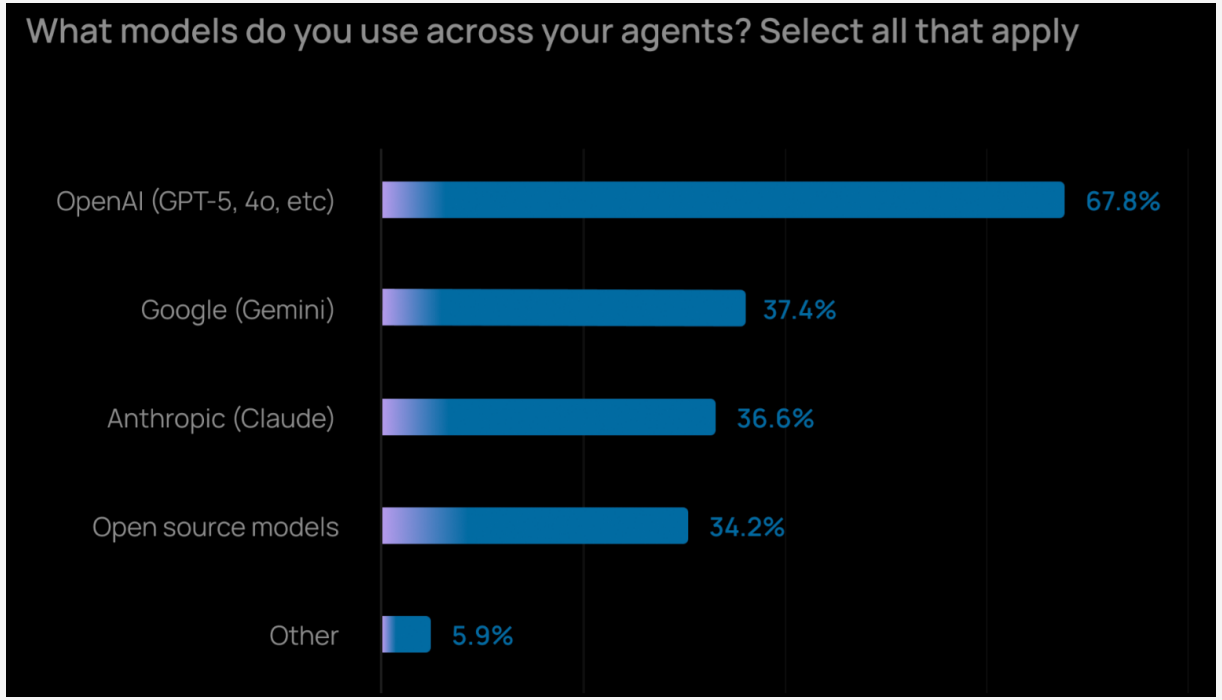
Requirement already satisfied: langchain in /usr/local/lib/python3.12/dist-packages (0.3.
Collecting langchain
  Downloading langchain-1.0.3-py3-none-any.whl.metadata (4.7 kB)
Collecting langchain-core<2.0.0,>=1.0.0 (from langchain)
  Downloading langchain_core-1.0.2-py3-none-any.whl.metadata (3.5 kB)
Collecting langgraph<1.1.0,>=1.0.2 (from langchain)
  Downloading langgraph-1.0.2-py3-none-any.whl.metadata (7.4 kB)
Requirement already satisfied: pydantic<3.0.0,>=2.7.4 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: jsonpatch<2.0.0,>=1.33.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: langsmith<1.0.0,>=0.3.45 in /usr/local/lib/python3.12/dist
```

출처 : https://docs.langchain.com/oss/python/langchain/overview

랭체인 시작하기

- Model Providers





출처 : https://www.langchain.com/state-of-agent-engineering#model-and-tool-landscape

랭체인 시작하기

- 언어 모델 Provider 패키지 설치
  - Open AI

```
pip install -U langchain-openai
```

  - Anthropic

```
pip install -U langchain-anthropic
```

  - Google

```
pip install -U langchain-google-genai
```

  - Ollama

```
pip install -U langchain-ollama
```

랭체인 시작하기

- API 키 os 환경 변수 저장
  - 모델 호출 시 API 키를 매번 입력하지 않아 코드가 간결해짐.

```
import os

os.environ["OPENAI_API_KEY"] = " sk-proj-..."
```

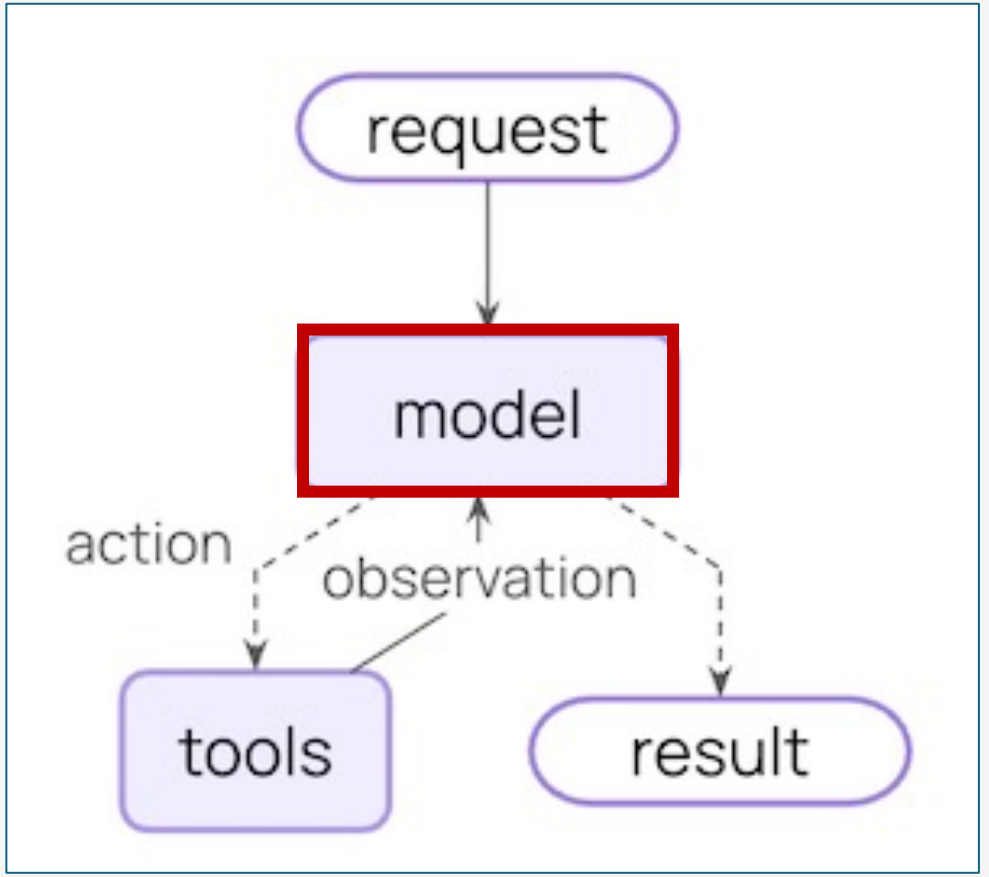
제 2장.

Model

2-1 Model I/O

Model I/O

1. 텍스트 생성 (Text Generation)
   - 사람처럼 텍스트 생성
2. 구조화된 답변 생성 (Structured Output)
   - JSON, 리스트, XML
3. 멀티 모달 (Multimodality)
   - 이미지, 영상, 음성 생성
4. 추론
5. Tool Calling



출처 : https://docs.langchain.com/oss/python/langchain/agents

Model I/O

1. `init_chat_model` 모듈을 가져와 모델 설정
2. `invoke()` 메서드를 사용해 답변(텍스트) 생성

```
from langchain.chat_models import init_chat_model


model = init_chat_model("gpt-5.4-mini")
response = model.invoke("안녕하세요. 당신은 누구십니까?")
print(response.content)

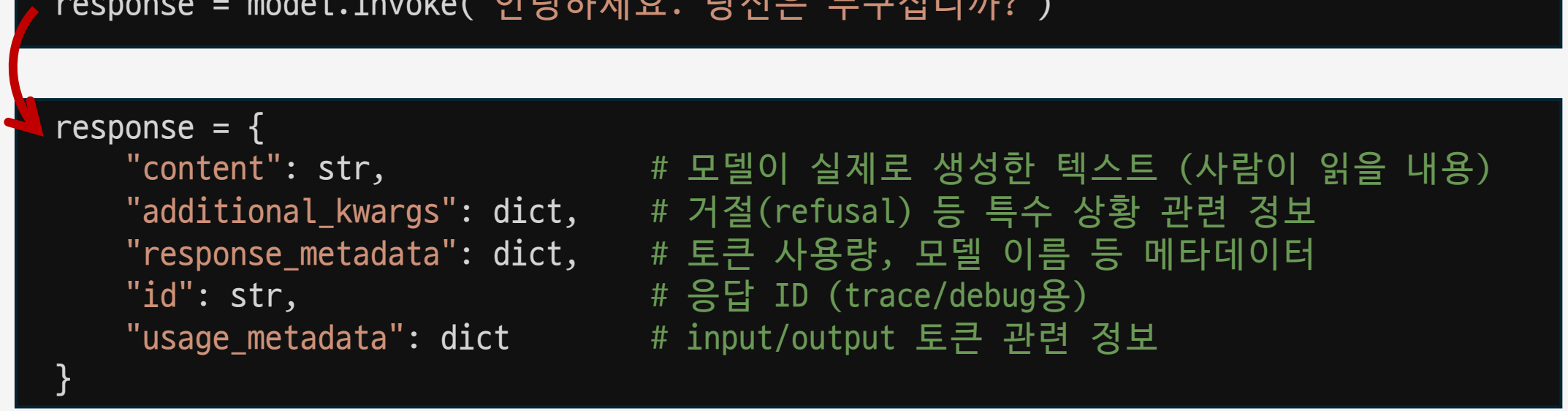
"""
안녕하세요! 저는 OpenAI가 만든 대화형 인공지능, GPT5-5 기반의 챗봇입니다.
"""
```

Model I/O

1. init_chat_model 모듈을 가져와 모델 설정
2. invoke() 메서드를 사용해 답변(텍스트) 생성

```python
from langchain.chat_models import init_chat_model

model = init_chat_model("gpt-5.4-mini")
response = model.invoke("안녕하세요. 당신은 누구십니까?")
```



```python
response = {
    "content": str,                # 모델이 실제로 생성한 텍스트 (사람이 읽을 내용)
    "additional_kwargs": dict,     # 거절(refusal) 등 특수 상황 관련 정보
    "response_metadata": dict,     # 토큰 사용량, 모델 이름 등 메타데이터
    "id": str,                     # 응답 ID (trace/debug용)
    "usage_metadata": dict         # input/output 토큰 관련 정보
}
```

Model I/O

- 모델 답변 (AIMessage 객체) 구조
  - usage_metadata
    - input_tokens : 입력 토큰 (안녕하세요 당신은 누구십니까?)
    - output_tokens : 응답 토큰
    - reasoning : 모델이 최종 텍스트를 생성하기 전, 혼자 속으로 논리를 전개하고 생각하는 데 사용한 토큰

```
{
    'input_tokens': 15,
    'output_tokens': 62,
    'total_tokens': 77,
    'input_token_details': {'audio': 0, 'cache_read': 0},
    'output_token_details': {'audio': 0, 'reasoning': 1664}
}
```

Model I/O

1. init_chat_model 모듈을 가져와 모델 설정

- model (필수) : 이름이나 모델의 ID → 'gpt-5-nano', 'claude-sonnet-4-5-20250929'
  - '{model_provider}:{model}' 형태로 모델 provider를 특정할 수 있음 → 'openai:o1'

```
init_chat_model(
    model: str | None = None,
    *,
    model_provider: str | None = None,
    configurable_fields: Literal["any"] | list[str] | tuple[str, ...] | None = None,
    config_prefix: str | None = None,
    **kwargs: Any,
) -> BaseChatModel | _ConfigurableModel
```

2-2 parameter

parameter

- init_chat_model 모듈 파라미터

```python
from langchain.chat_models import init_chat_model

model = init_chat_model(
    "claude-sonnet-4-5-20250929",
    temperature=0.5,
    timeout=10,
    max_tokens=1000
)
```

Parameters

A chat model takes parameters that can be used to configure its behavior. The full set of supported parameters varies by model and provider, but standard ones include:

model `string` `required`

The name or identifier of the specific model you want to use with a provider. You can also specify both the model and its provider in a single argument using the ':' format, for example, 'openai:o1'.

api_key `string`

The key required for authenticating with the model's provider. This is usually issued when you sign up for access to the model. Often accessed by setting an environment variable.

temperature `number`

Controls the randomness of the model's output. A higher number makes responses more creative; lower ones make them more deterministic.

max_tokens `number`

Limits the total number of tokens in the response, effectively controlling how long the output can be.

timeout `number`

The maximum time (in seconds) to wait for a response from the model before canceling the request.

max_retries `number`

The maximum number of attempts the system will make to resend a request if it fails due to issues like network timeouts or rate limits.

parameter

- `init_chat_model` 모듈 파라미터
  - temperature (창의성)
    - 답변의 무작위성을 조절하여 **창의적**(High) 혹은 **논리적**(Low) 성향 결정 (0~1.0)
  - max_tokens (용량)
    - 응답의 **최대 길이를 제한**하여 비용 관리 및 출력 통제 (토큰)
  - timeout (시간)
    - 응답 대기 시간을 제한하여 시스템의 **무한 대기**(Hang) **방지** (초)
  - max_retries (안정성)
    - 네트워크 오류 발생 시 **자동 재시도**를 통해 연결 성공률 확보

parameter

- OpenAI 특화 파라미터
  - reasoning_effort : 모델이 최종 텍스트를 생성하기 전,
    혼자 속으로 논리를 전개하고 생각하는 데 얼마나 노력할지 설정
    - none : 추론력 비활성화
    - low : 약간의 추가 사고
    - medium : 계획 수립 약간 복잡한 추론
    - high : 코딩같이 매우 복잡한 추론

```
model = init_chat_model(
    "gpt-5.4-mini",
    temperature = 1.2,
    reasoning={
        "effort":  " low", # low / medium / high
    }
)
```

parameter

- OpenAI 특화 파라미터
  - reasoning_effort : 모델이 최종 텍스트를 생성하기 전,

    혼자 속으로 논리를 전개하고 생각하는 데 얼마나 노력할지 설정

```
response.content
```


```
[{'id': 'rs_0cba82419e47e3970069c0e6b251fc819793d873c60de5fe17',
  'summary': [],
  'type': 'reasoning'},
 {'type': 'text',
  'text': '실시간 날씨를 제가 직접 조회하진 못해서 **오늘 서울의 정확한 기온/강수 여부**를 지금 바로 확인해드리긴 어려워요. 대신 가장 빠르게 확인하는 방법은 아래예요.\n\n- **네이버/다음 “서울 날씨”** 검색  \n- **기상청(날씨누리)** 또는 **기상청 앱**  \n- **스마트폰 날씨 위젯**(예: 삼성/애플 기본 날씨)\n\n원하시면, **지금 계신 동네(예: 강남구/마포구 등)**랑 “출퇴근 시간대” 정도를 알려주시면, 확인할 때 무엇을 중점으로 보면 좋은지(예: 우산 필요 여부, 체감온도, 미세먼지) 체크 포인트로 정리해드릴게요.',
  'annotations': [],
  'id': 'msg_0cba82419e47e3970069c0e6b295188197a4e8fb0fee41e777'}]
```


👉 결과 예시

```
response.text
```

2-3 주요 메서드

주요 메서드

- stream( ) : chunk(말뭉치) 단위로 잘라서 하나씩 답변 생성 → UX 증가

```
for chunk in model.stream("AI Agent란 무엇인지 1000자 이상으로 설명해 주세요"):
    print(chunk.text, end="")
```

- batch( ) : 여러 질문을 병렬적으로 요청 가능 → UX 증가 및 모델 호출을 줄여 비용 감소

```
responses = model.batch([
    "과적합이 뭔가요?",
    "앵무새의 털 색상이 화려한 이유는?",
    "AI Agent 기반 서비스는 뭐가 있나요?"
])

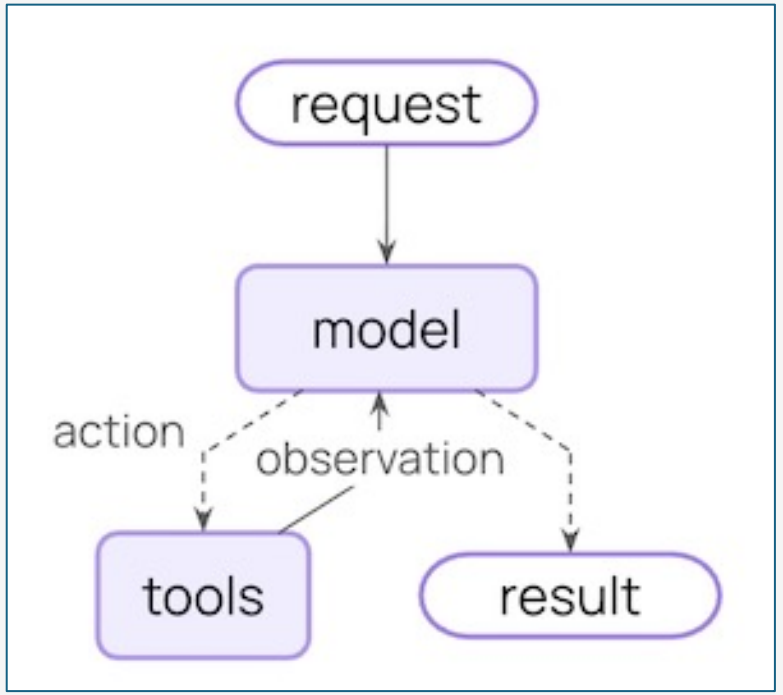
for response in responses:
    print(response)
```

2-4 메모리 구현

메모리 구현

- 메시지 구성 요소
  - System Message : 개발자가 사전에 작성한 입력 (프롬프트)
  - Human Message : 사용자의 요청(request), 입력
  - AI Message : Model의 답변 (result)

```
[
    SystemMessage("당신은 유능한 로켓 전문가 입니다.")
    HumanMessage("안녕하세요. 궁금한 게 있어요!")
    AIMessage( " 로켓의 기본 원리나 추진 발사 차이 등...")
]
```



출처 : https://docs.langchain.com/oss/python/langchain/messages

메모리 구현

- 예시 코드 1 – Message 객체 사용

```python
from langchain.chat_models import init_chat_model
from langchain.messages import HumanMessage, SystemMessage


model = init_chat_model("gpt-5.4-mini")


system_msg = SystemMessage("당신은 유능한 로켓 전문가입니다.")
human_msg = HumanMessage("안녕하세요. 궁금한 게 있어요")

messages = [system_msg, human_msg]
response = model.invoke(messages)    # AIMessage 반환


print(response)  # content='로켓의 기본 원리나 추진 발사 차이 등... '
```

출처 : https://docs.langchain.com/oss/python/langchain/messages

메모리 구현

- 예시 코드 2 – 딕셔너리 포맷 (OpenAI)
  - user - human / ai - assistant

```
messages = [
    {"role": "system", "content": "당신은 유능한 AI 어시스턴트입니다."},
    {"role": "user", "content": "안녕하세요. 저는 Jay입니다!"},
    {"role": "assistant", "content": "안녕하세요 Jay! 어떤 도움이 필요하신가요?"},
    {"role": "user", "content": "제 이름이 뭐라고 했죠?"},
]


response = model.invoke(messages)


print(response)  # content='당신은 Jay입니다...'
```

출처 : https://docs.langchain.com/oss/python/langchain/messages

2-5 구조화된 답변

구조화된 답변

- Structured Outputs
  - 다음 작업을 위해 답변을 특정 형태로 **파싱**할 때 사용
    1. 스키마 (class) 생성
    2. model에 `with_structured_output`을 사용해 새로운 모델 객체 생성

```python
from pydantic import BaseModel, Field

class Movie(BaseModel):
    """상세한 영화 정보."""
    title: str = Field(description="영화의 제목")
    year: int = Field(description="개봉 연도")
    director: str = Field(description="영화 감독 이름")
    rating: float = Field(description="영화 평점 (10점 만점)")

model_with_structure = model.with_structured_output(Movie)
response = model_with_structure.invoke("영화 인셉션에 대해 설명해 주세요")

# Movie(title='인셉션', year=2010, director='크리스토퍼 놀란', rating=8.8)
```

구조화된 답변

- Structured Outputs
  - 스키마 생성

1. pydantic : 데이터 검증(Validation), 직렬화(Serialziation)

```
Requirement already satisfied: pydantic<3.0.0,>=2.7.4 in /usr/local/lib/python3.12/dist-p
```

1.클래스 선언 (`class Movie(BaseModel)`)
2.필드 정의 (`title : str`)
3.기본값/Field 설정 (`= Field(...)`)
4.docstring (`"상세한 영화 정보."`) : 모델 전체에 대한 설명

```python
from pydantic import BaseModel, Field

class Movie(BaseModel):
    """상세한 영화 정보."""
    title: str = Field(description="영화의 제목")
```

구조화된 답변

- Structured Outputs
  - 스키마 생성
    2. JSON Schema
       - 영어/숫자/언더스코어/대시만 사용 가능
       - pydantic은 파싱 + 데이터 검증까지 지원, JSON Schema는 파싱만 지원

```
import json


json_schema = {
    "title": "Movie",
    "description": "A movie with details",
    "type": "object",
    "properties": {
        "title": {
            "type": "string",
            "description": "The title of the movie"
        },
        "year": {
            "type": "integer",
            "description": "The year the movie was released"
        },
    },
    "required": ["title", "year", "director", "rating"]
}
```

출처 : https://platform.openai.com/docs/guides/structured-outputs

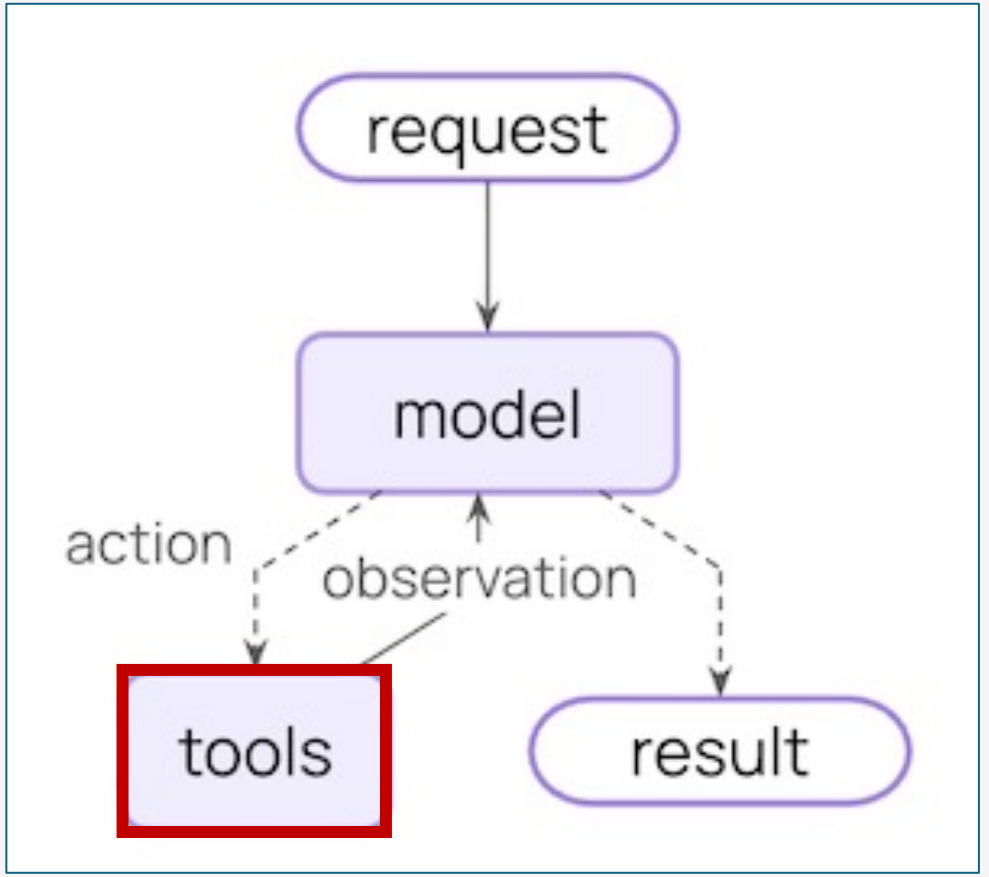
제 3장.

Agents

3-1 Tool 기초

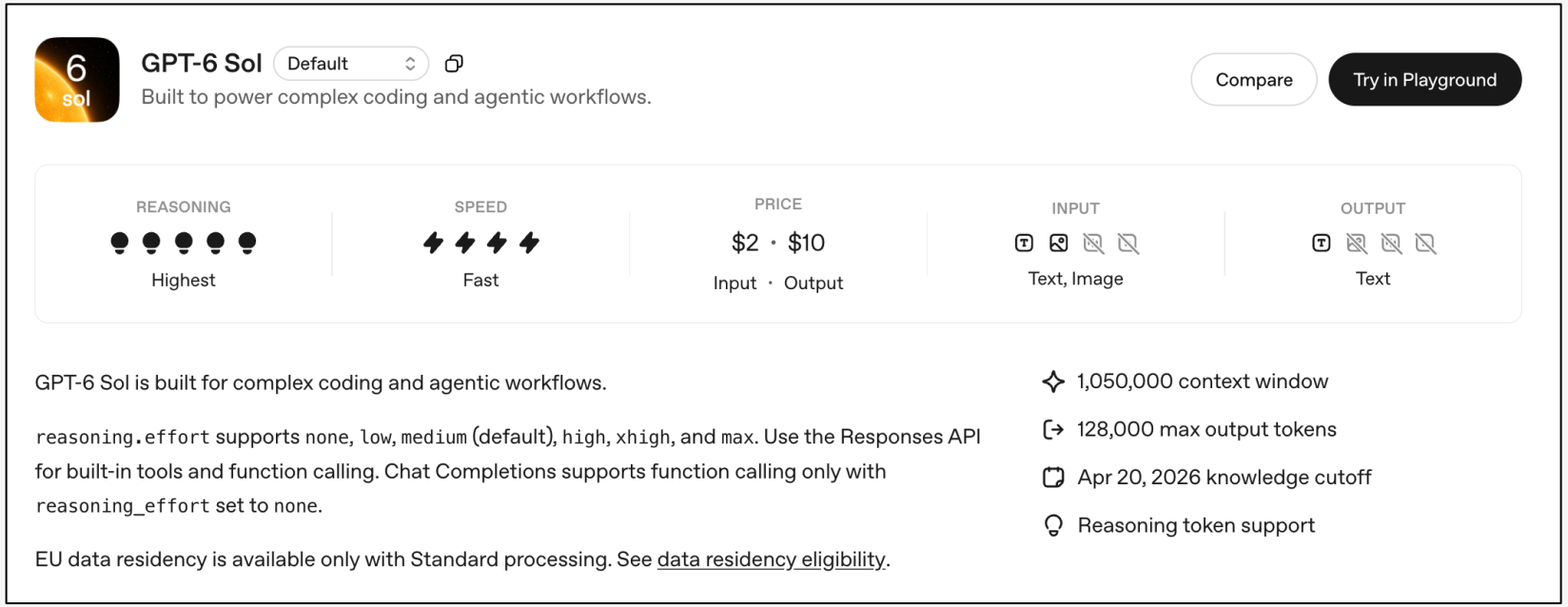
(복습) 랭체인이란?

- Agent
  - 사용자의 입력(질문, 지시)에 대해 적절한 행동을 취하도록 **추론(Reasoning)**한 뒤 **행동(Act)**하는 **주체(Excutor)**
  - 구성 요소
    - Model (LLM)
    - Tools (함수)



출처 : https://docs.langchain.com/oss/python/langchain/agents

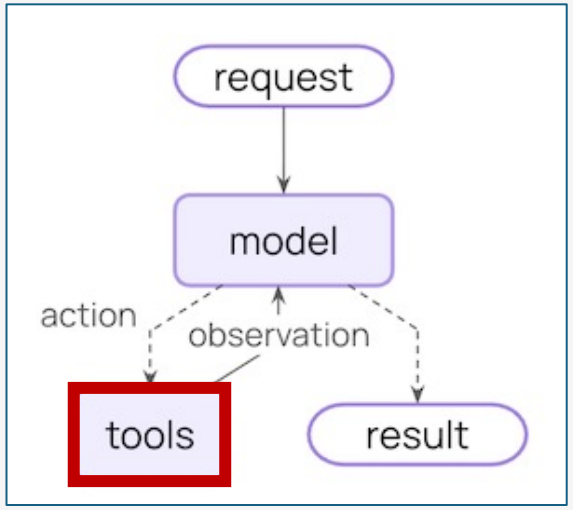
Tool 기초



현재 GPT-6-Sol의 경우 KC가 2026년 4월 20일...

지금 서울 날씨를 물어보면 모델은 답변할 수 있을까?

Tool 기초



날씨 관련 요청이 들어오면 model이 날씨를 검색하는 tool을 호출하고,

tool의 검색 결과를 참고해 model이 답변하면 어떨까?

출처 : https://platform.openai.com/docs/models/gpt-5-nano

Tool 기초

- Tool
  - 모델(LLM)이 **외부 시스템(API, DB, 파일 시스템 등)**과 상호작용하기 위해 사용하는 요소
  - 구성 요소 : ① **호출 가능한 함수**(callable function) + ② 입력 스키마(input schema)

→ Tool은 함수다!!!

Tool 기초

- Tool
  - 모델(LLM)이 **외부 시스템(API, DB, 파일 시스템 등)**과 상호작용하기 위해 사용하는 요소
  - 구성 요소 : ① **호출 가능한 함수**(callable function) + ② 입력 스키마(input schema)
  - **Tool 작성법**

1. 함수 작성
2. 타입 힌팅
3. docstring으로 함수(Tool) 설명
4. @tool 데코레이터 입력

```python
from langchain.tools import tool

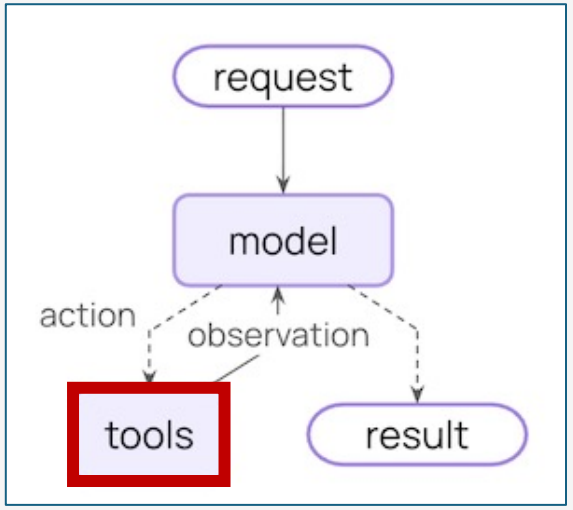
@tool
def get_weather(location: str) -> str:
    """특정 지역의 날씨 정보 제공"""
    return f"{location}의 날씨는 맑고, 영하 2도입니다"
```

Tool 기초

- Tool – Model 연결 (Agent 객체 생성)
  - langchain.agents 의 create_agent를 사용해 모델과 tools을 전달하면 **Agent 생성**

```
from langchain.agents import create_agent

agent = create_agent(model, tools=[get_weather])
```



Tool 기초

- Tool – Model 연결 (Agent 객체 생성)
  - invoke( ) 메서드로 Agent 실행
  - messages에 담긴 리스트 속 마지막 content (result["messages"][-1].content)에 최종 응답 확인

```
result = agent.invoke(
    {"messages": [{"role": "user", "content": ”현재 한국 날씨 어때?"}]},
)

print(result["messages"][-1].content)
# 현재 한국 날씨는 맑고 영하 2도예요. 상당히 추으니 외출 시 두꺼운 옷이나...
```

→ 응답(result)는 어떻게 생성된걸까?

Tool 기초

- Tool – Model 연결 (Agent 객체 생성)

```
{
    "messages": [
        HumanMessage(content='현재 한국 날씨 어때요?', ...),
        AIMessage(content='', tool_calls=[{'name': 'get_weather', 'args': {'location': '한국'}}]),
        ToolMessage(content='한국의 날씨는 맑고, 영하 2도입니다', name='get_weather', ...),
        AIMessage(content='현재 한국 날씨는 맑고 영하 2도예요. 상당히 추우니...')
    ]
}
```

- Model이 content를 생성하지 않고, tool_calls로 get_weather 도구 호출
- 도구는 응답을 ToolMessage에 담아 응답
- Model은 도구의 응답을 참조해 답변 생성

Tool 기초

```
{
    "messages": [
        HumanMessage(content='현재 한국 날씨 어때요?', ...),
        AIMessage(content='', tool_calls=[{'name': 'get_weather', 'args': {'location': '한국'}}]),
        ToolMessage(content='한국의 날씨는 맑고, 영하 2도입니다', name='get_weather', ...),
        AIMessage(content='현재 한국 날씨는 맑고 영하 2도예요. 상당히 추우니...')
    ]
}
```

Tool을 호출할지 말지, 어떤 Tool을 호출할지,

호출한다면 Tool 호출에 필요한 인자 ({'location' : '서울'})는 누가 작성한걸까?

→ Agent 객체 속 AI Model

Tool 기초

- Agent 작동 순서
  1. HumanMessage(〃현재 한국 날씨 어때요?")
  2. Model (추론) : "이건 get_weather를 호출해야겠군"
  3. AIMessage(content='', tool_calls = [{"name": "get_weather", "args": {"location": 〃한국"}}])
  4. get_weather( ) 실행
  5. ToolMessage(〃한국의 날씨는 맑고 영하 2도입니다")
  6. AIMessage(최종 응답 생성): "현재 한국 날씨는 맑고 영하 2도예요..."

[실습] Tool 기초

- 실습 1

Python의 내장 라이브러리인 datetime을 활용하여,

현재 날짜(월, 일)와 시간(시, 분)을

정확히 응답할 수 있는 에이전트를 구축하세요

[실습] Tool 기초

- 실습 1

```
result = agent.invoke(
    {"messages": [{"role": "user", "content": "오늘이 몇 월 며칠이고 지금이 몇 시 몇 분이야?"}]},
)
```

```
result['messages'][-1].content
```

```
'오늘은 2026년 3월 10일이고, 지금은 17시 45분입니다. (오후 5시 45분)'
```

👉 결과 예시

3-2 Tool 응용

Tool 응용

- Tool 정의

```python
from langchain.tools import tool

@tool
def add(a: int, b: int) -> int:
    """`a`와 `b` 덧셈.

    Args:
        a: First int
        b: Second int
    """
    return a + b

@tool
def multiply(a: int, b: int) -> int:
    """`a`와 `b` 곱셈.

    Args:
        a: First int
        b: Second int
    """
    return a * b

@tool
def divide(a: int, b: int) -> float:
    """`a`와 `b` 나눗셈.

    Args:
        a: First int
        b: Second int
    """
    return a / b
```

- Agent 생성

```
tools = [add, multiply, divide]


agent = create_agent(model,tools)
```

Tool 응용

- Agent 실행

```
{
    "messages": [
        HumanMessage(content='42 + 3 * 23은 뭔가요?', ...),
        AIMessage(content='111입니다.\n\n계산 흐름: 곱셈이 먼저니까 3 × 23 ...', ...),
    ]
}
```

→ Tool Calling 없이 바로 답변 생성

Tool 응용

- System Message in Agents
  - create_agent에 system_prompt 입력 → SystemMessage 객체로 자동 전환

```
agent = create_agent(
    model,
    tools,
    system_prompt="당신은 유능한 수학 선생님입니다. 계산 시 모든 단계에서 tool들을 이용하시오."
)
```

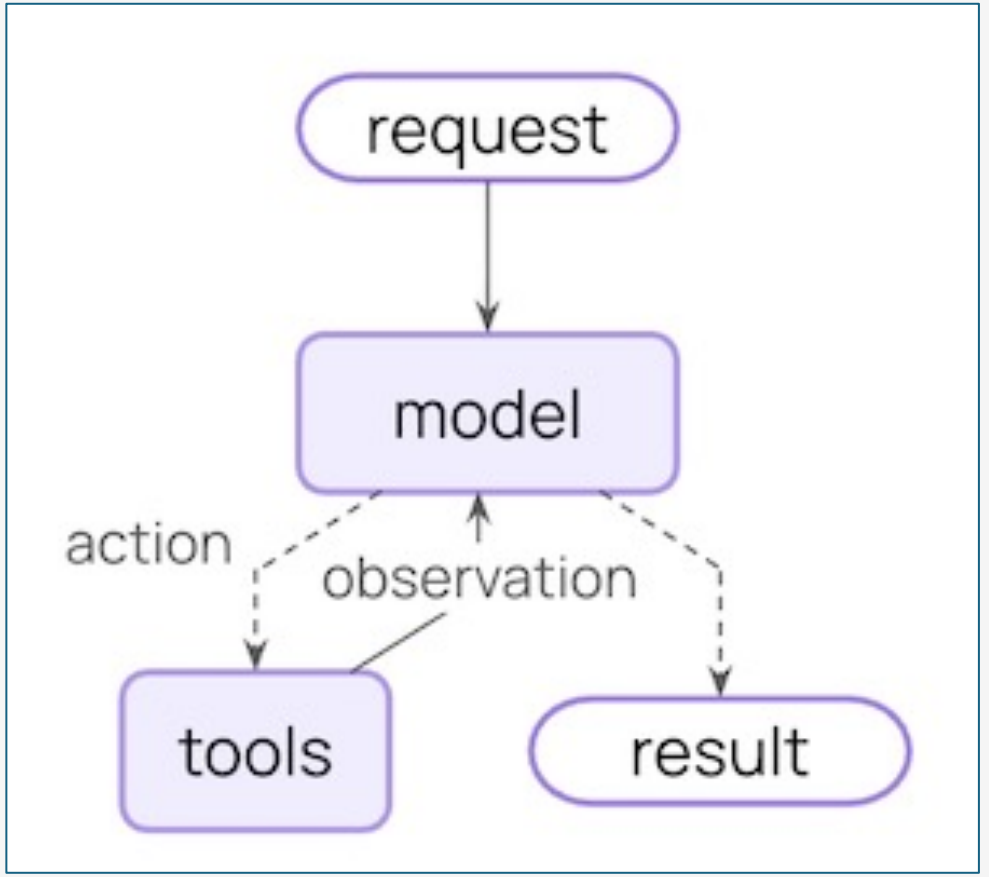
Tool 응용

- Agent 실행 결과

```
{
    "messages": [
        HumanMessage(content='42 + 3 * 23은 뭔가요?', ...),
        AIMessage(content='', tool_calls=[{'name': 'multiply', 'args': {'a': 3, 'b': 23}}],
        ToolMessage(content= '69', name= 'multiply', ...),
        AIMessage(content='', tool_calls=[{'name': ‘add', 'args': {'a’: 42, 'b’: 69}}],
        ToolMessage(content= ‘111', name= ‘add', ...),
        AIMessage(content='', tool_calls=[{'name': ‘add', 'args': {'a’: 42, 'b’: 69}}],
        ToolMessage(content= ‘111', name= ‘add', ...),
        AIMessage(content='정답은 111입니다.\n\n설명:\n- 먼저 곱셈의...’)
    ]
}
```

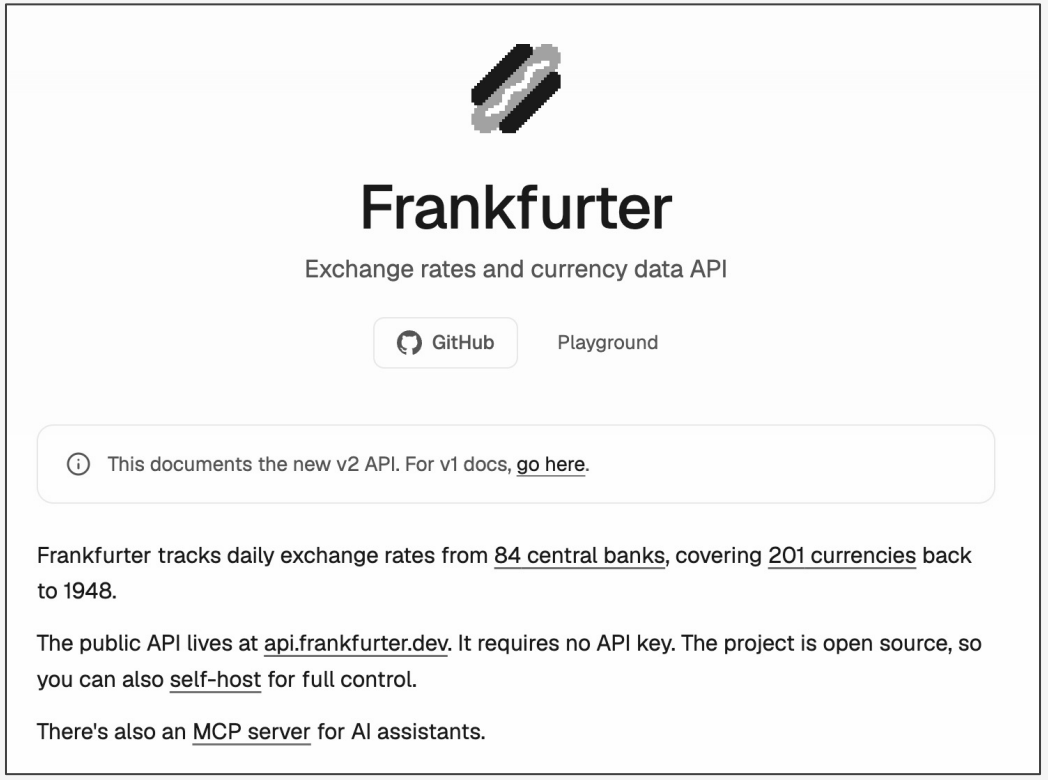
Tool 응용

- Agent 실행 결과
  1. 사용자 질문
  2. [AI] 곱셈 필요 판단 → multiply(3, 23)
  3. [Tool] multiply 결과: 69
  4. [AI] 덧셈 필요 판단 → add(42, 69)
  5. [Tool] add 결과: 111
  6. [AI] (내부 reasoning 반복 시도)
  7. [AI] 최종 답변 출력: "정답은 111입니다."



Tool 응용

- Tool 응용 실습
  - 환율 API (https://frankfurter.dev/) 를 사용해 원-달러 환율을 답변하는 Tool을 만들고, 오늘의 원-달러 환율을 질문해 답변을 받으시오.



Tool 응용

- Tool 응용 실습
  - 환율 API (https://frankfurter.dev/) 를 사용해 원-달러 환율을 답변하는 Tool을 만들고, 오늘의 원-달러 환율을 질문해 답변을 받으시오.

```
result = agent.invoke(
    {
        "messages": [
            {
                "role": "user",
                "content": "오늘 원-달러 환율 알려줘",
            }
        ]
    }
)
```

```
result["messages"][-1].content
```

```
'오늘 기준 원-달러 환율은 **1달러 = 1,483.78원**입니다.\n\n- **기준일:** 2026-07-19\n- **1달러당 원화 환율:** 1,483.78 KRW\n- **Frankfurter API 기준:** 조회된 환율 정보는 Frankfurter API 기준입니다.'
```

Tool 응용



〈ChatGPT 속 Tool 호출〉

3-3 Built-in Tools

Built-in Tools

랭체인에 이미 많은 Tool들이 통합 되어있다!

(링크)

Built-in Tools

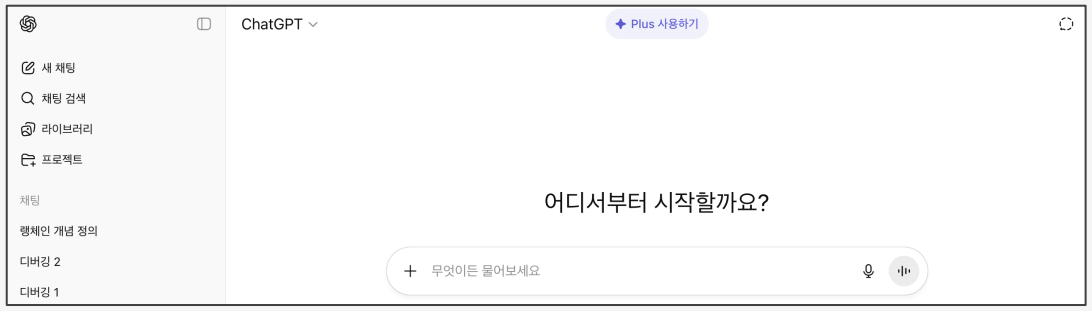
- Python REPL
  - https://docs.langchain.com/oss/python/integrations/tools/python

- Tavily search
  - https://docs.langchain.com/oss/python/integrations/tools/tavily_search

3-4 Agent 메모리

Agent 메모리

- 다중 턴 대화 시 대화 맥락을 고려한 Tool 호출을 위해 메모리 관리 필요
- 메모리 종류
  - Short-term memory : Checkpointer를 통해 대화 스레드(세션) 저장
  - Long-term memory : Store를 통해 장기 메모리 저장





출처 : https://docs.langchain.com/oss/python/langgraph/add-memory#add-short-term-memory

Agent 메모리

- Short-term memory
  - Checkpointer : 스레드(대화 상태)를 저장할 저장소
  - checkpointer=InMemorySaver( ) → 기억용 수첩과 펜을 Agent에게 쥐여주기

```
from langgraph.checkpoint.memory import InMemorySaver


agent = create_agent(
    model,
    tools,
    checkpointer=InMemorySaver(),
)
```

Agent 메모리

- Short-term memory
  - 대화 저장 1
    - thread_id 1에 첫 번째 대화 저장

```
response = agent.invoke(
    {"messages": [{"role": "user", "content": "안녕하세요! 저는 홍길동입니다."}]},
    {"configurable": {"thread_id": "1"}},
)
```

```
response = agent.invoke(
    {"messages": [{"role": "user", "content": "안녕하세요! 저는 홍길동 입니다."}]},
    {"configurable": {"thread_id": "1"}},
)
```

```
response["messages"][-1].content
```

```
'안녕하세요, 홍길동님! 😊  \n만나서 반갑습니다. 오늘 어떤 도움 드릴까요?'
```

Agent 메모리

- Short-term memory
  - 대화 저장 2
    - thread_id 1에 두 번째 대화 저장

```
response = agent.invoke(
    {"messages": [{"role": "user", "content": "방금 뭐라고 했죠 제가?"}]},
    {"configurable": {"thread_id": "1"}},
)
```

```
response = agent.invoke(
    {"messages": [{"role": "user", "content": "방금 뭐라고 했죠 제가?"}]},
    {"configurable": {"thread_id": "1"}},
)
```

```
response["messages"][-1].content
```

```
'방금 이렇게 말씀하셨어요: **"안녕하세요! 저는 홍길동 입니다."**'
```

Agent 메모리

- Short-term memory
  - 대화 내역 출력
    - thread_id를 바꾸면 새로운 대화 쓰레드 생성

```
response = agent.invoke(
    {"messages": [{"role": "user", "content": "지금까지 무슨 얘기 나눴죠?"}]},
    {"configurable": {"thread_id": "2"}},
)
```

```
for i, msg in enumerate(response["messages"]):
    print(f"--- Message {i+1} ---")
    print(msg.content)
    print()
```

```
--- Message 1 ---
지금까지 무슨 얘기 나눴죠?

--- Message 2 ---
지금까지의 대화 내용을 제가 직접 보지 못해서 요약해 드리려면 두 가지가 필요합니다.

- 방법 1: 지금까지의 대화 내용을 이 창에 붙여넣어 주기
- 방법 2: 기억하고 싶은 포인트나 키워드를 알려 주기(예: 핵심 주제, 결정된 사항, 남은 질문, 다음 단계)

원하시는 요약 형식도 알려 주세요:
- 짧은 요약(1~2문장)
- 포인트별 상세 요약
- 의사결정 목록/다음 단계

어떤 형식으로 요약해 드릴지 알려 주시겠어요? 또는 대화 내용을 붙여 주시면 바로 요약해 드리겠습니다.
```

3-5 구조화된 답변 생성

(복습) 구조화된 답변 생성

- Structured Outputs
  - 다음 작업을 위해 답변을 특정 형태로 **파싱**할 때 사용
    1. 스키마 (class) 생성
    2. model에 `with_structured_output`을 사용해 새로운 모델 객체 생성

```python
from pydantic import BaseModel, Field

class Movie(BaseModel):
    """상세한 영화 정보."""
    title: str = Field(description="영화의 제목")
    year: int = Field(description="개봉 연도")
    director: str = Field(description="영화 감독 이름")
    rating: float = Field(description="영화 평점 (10점 만점)")

model_with_structure = model.with_structured_output(Movie)
response = model_with_structure.invoke("영화 인셉션에 대해 설명해 주세요")

# title='인셉션' year=2010 director='크리스토퍼 놀란' rating=8.8
```

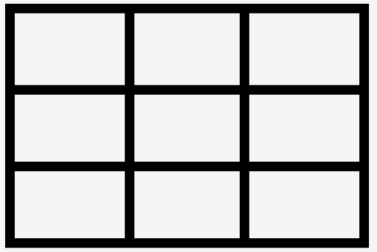
구조화된 답변 생성

- Structured Outputs
  - Agent의 실행 결과를 자연어가 아닌, 특정 포멧의 구조화된 데이터로 파싱 가능

    case 1) 고객의 컴플레인 메일을 Agent가 답장 생성 → 일반 자연어 답변 생성

    case 2) Agent가 메일의 의도, 감정 상태, 요약, 다음 수행 업무 등을 분석 후 구조에 맞춰 데이터 생성 → 구조화된 답변 생성





구조화된 답변 생성

- Structured Outputs
  - Pydantic을 활용한 데이터 구조 정의

```python
from pydantic import BaseModel, Field
from typing import Literal


# Structured Output 정의
class EmailAnalysis(BaseModel):
    """이메일 내용을 분석한 결과 구조."""
    intent: Literal["complaint", "inquiry", "confirmation", "other"] = Field(
        description="이메일의 주요 의도 (예: complaint=불만, inquiry=문의, confirmation=확인, other=기타)"
    )
    sentiment: Literal["positive", "negative", "neutral"] = Field(description="이메일의 감정 상태")
    summary: str = Field(description="이메일 내용 요약")
    next_action: str = Field(description="에이전트가 수행해야 할 다음 단계 (예: 회신, 확인, 무시 등)")
```

구조화된 답변 생성

- Structured Outputs
  - ToolStrategy에 클래스 전달

```
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy


agent = create_agent(
    model="gpt-5.4-mini",
    tools=[],
    response_format=ToolStrategy(EmailAnalysis), # ToolStrategy에 구조 전달
)
```

구조화된 답변 생성

- Structured Outputs
  - 실행 결과 : 'structured_response'에 파싱 된 결과물 출력

```
Subject: 배송 지연 및 손상 상품에 대한 환불 요청 (주문번호 2024-000157)\n\n안녕하세요 고객센터 담당자님,\n\n온라인몰에서 2024-000157번 주문건에 대해 컴플레인을 남깁니다.
```

```
response["structured_response"]
```

```
EmailAnalysis(intent='complaint', sentiment='negative', summary='주문 2024-000157에 대한 배송 지연, 수령 박스 손상, 한쪽 이어폰 불량 및 충전 케이스 미충전 등으로 환불 또는 동일 모델 교환 요청. 반품 배송 라벨 제공 및 반품 방법 안내, 배송 지연에 대한 사과 및 보상 여부 문의.', next_action='고객에게 환불 또는 교환 옵션을 안내하고, 반품 라벨 발급 및 반품 절차를 제공하며, 배송 지연에 대한 사과와 보상 정책 여부를 안내합니다. 필요 시 주문 상태 확인 및 내부 조치를 진행.')
```

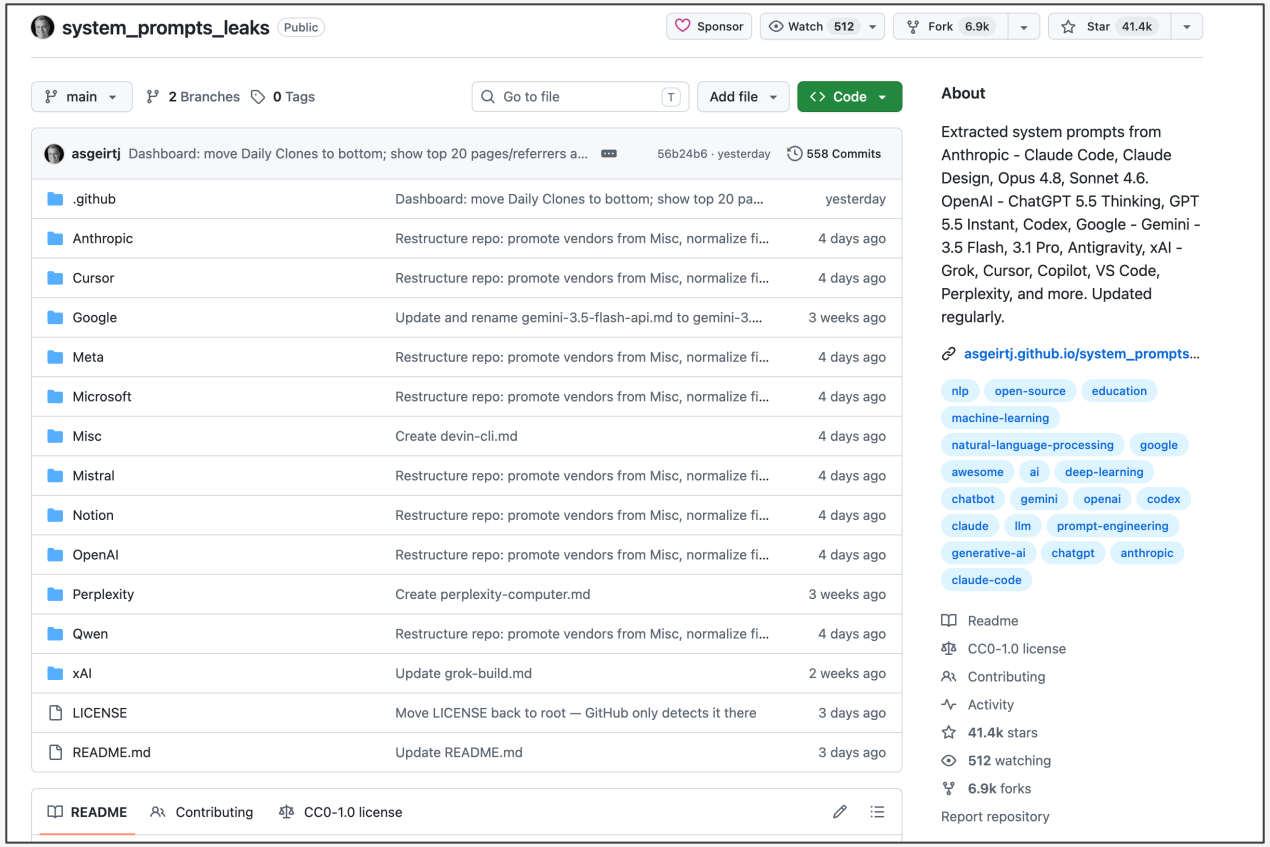
제 4장.

참고 자료

4-1 System Prompt

System Prompt

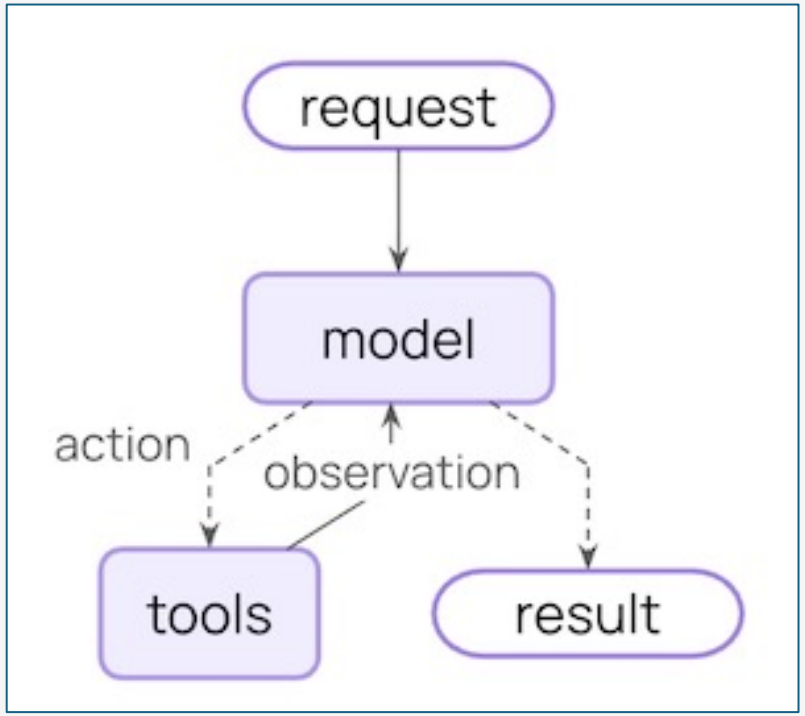
- Reverse Engineering 기반 시스템 프롬프트 (비공식)
  - https://github.com/asgeirtj/system_prompts_leaks



4-2 ReAct

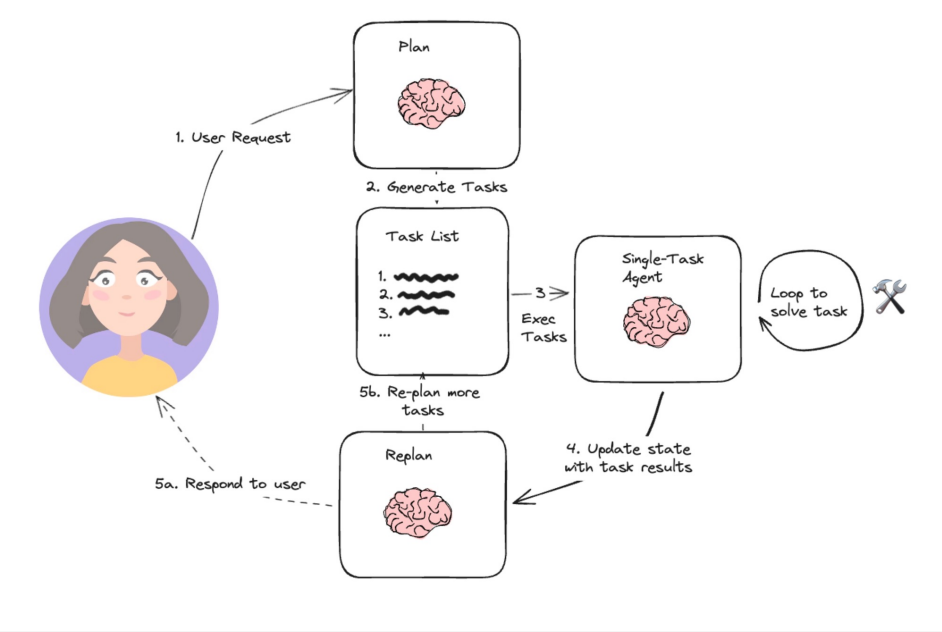
ReAct

- ReAct = Reasoning + Acting



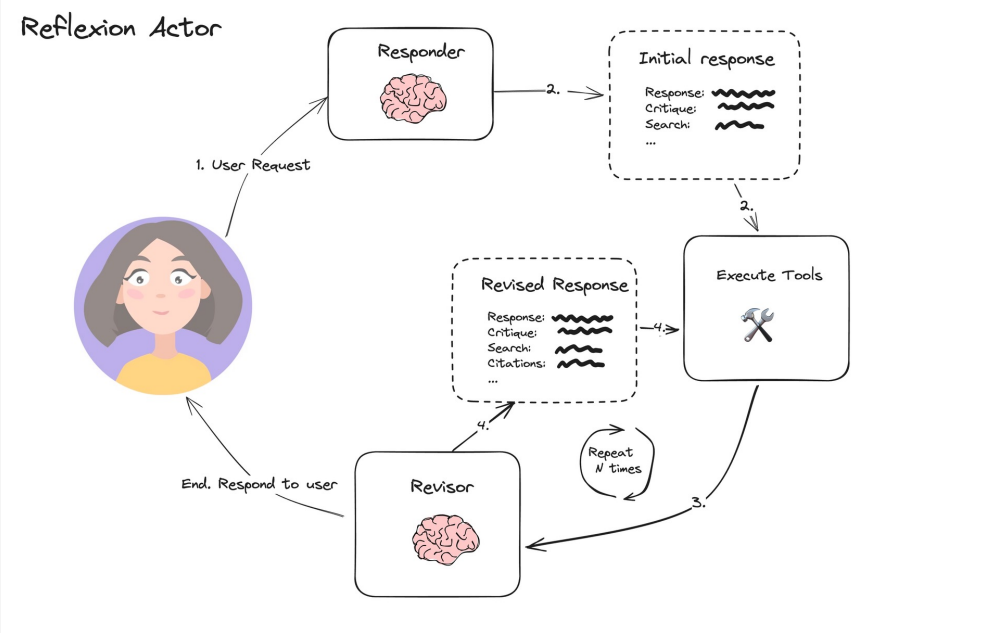
ReAct

- 다양한 Agent Design Pattern
  - Plan and Execute
    - 전체 계획 수립 → 단계별 실행 → 결과 정리
    - 복잡한 작업을 작은 하위 작업으로 먼저 나눠야 할 때 적합
    - https://www.langchain.com/blog/planning-agents



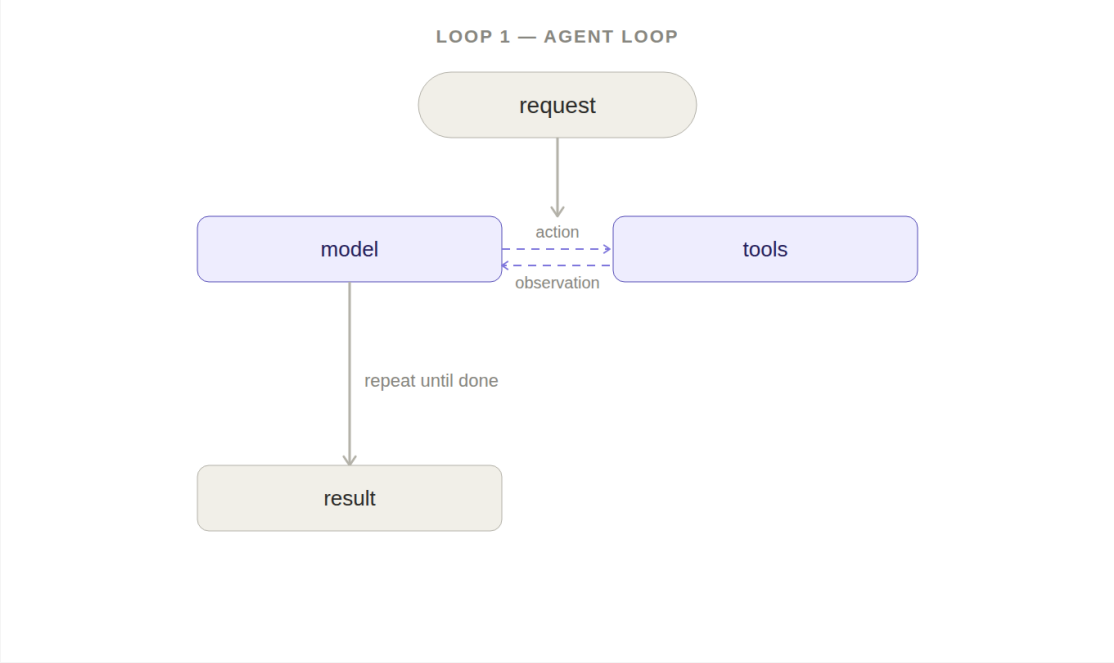
ReAct

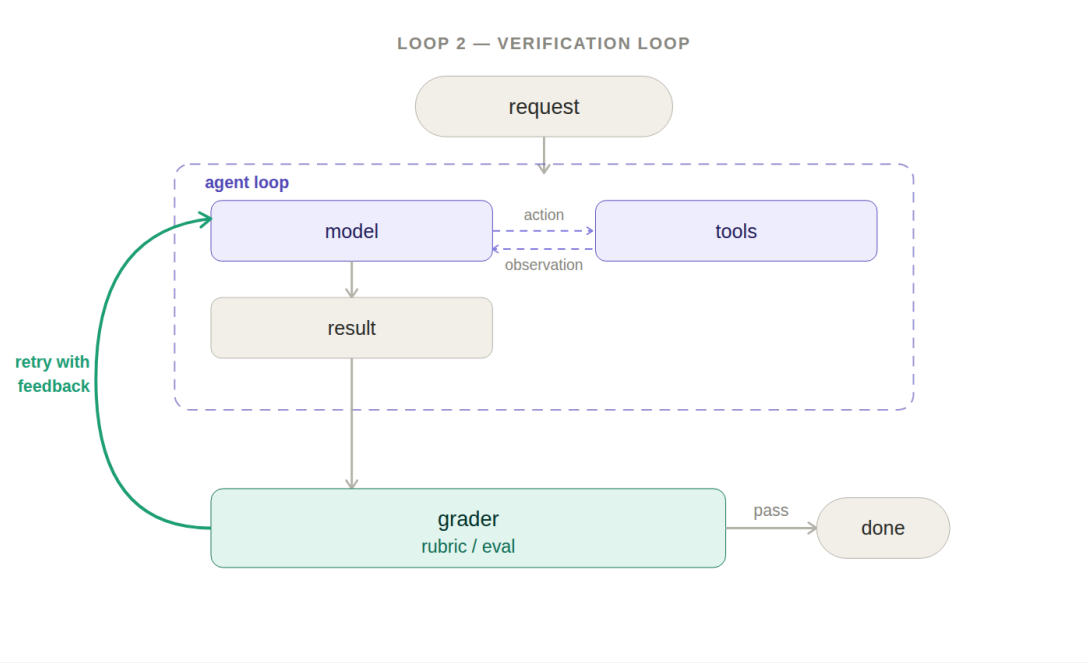
- 다양한 Agent Design Pattern
  - Reasoning Without Observation
    - 작업 수행 → 평가 → Self-Reflection → 개선된 전략으로 재시도
    - 에이전트가 실패 결과를 보고 스스로 반성한 뒤 다시 시도하는 구조
    - https://www.langchain.com/blog/reflection-agents



ReAct

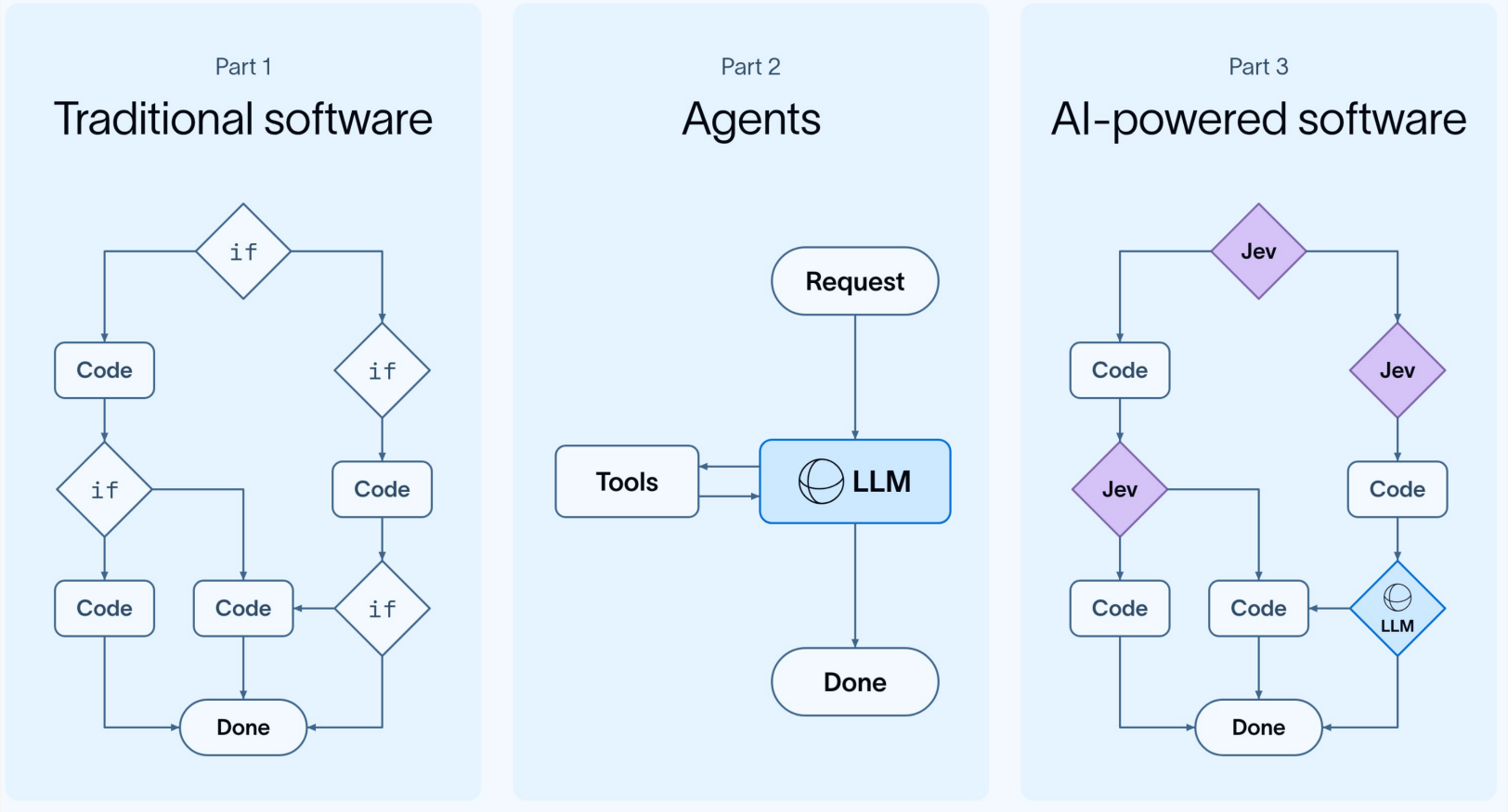
- Loop Engineering
  - Core Agent Algorithm : LLM이 업무를 완수할 동안 계속 루프를 돌게 설계하자!
  - https://www.langchain.com/blog/the-art-of-loop-engineering





ReAct

- Jev 기반 Agent

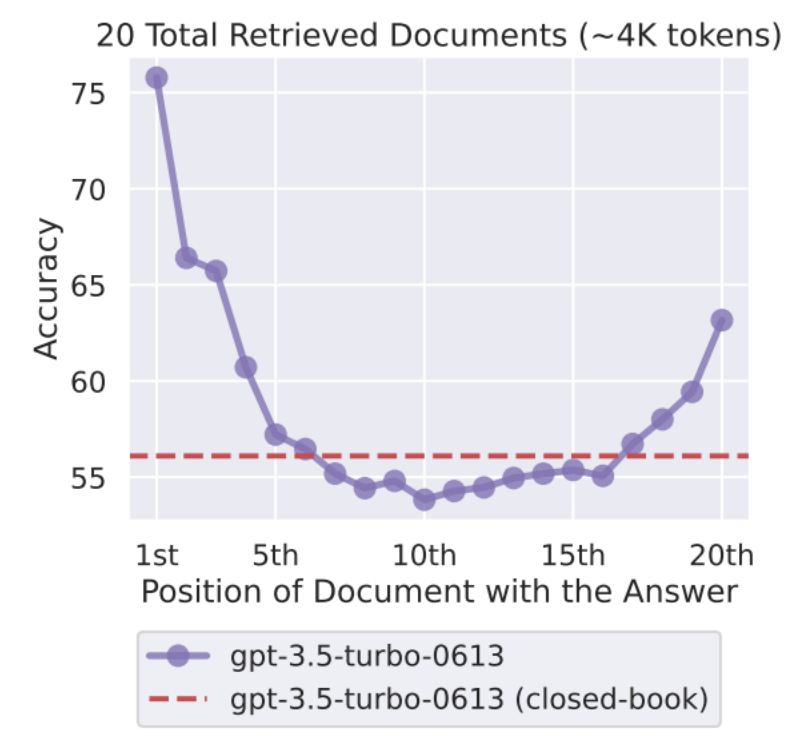


출처 : https://www.langchain.com/blog/building-prod-with-jev-and-langgraph

4-3 Lost in the Middle

Lost in the Middle

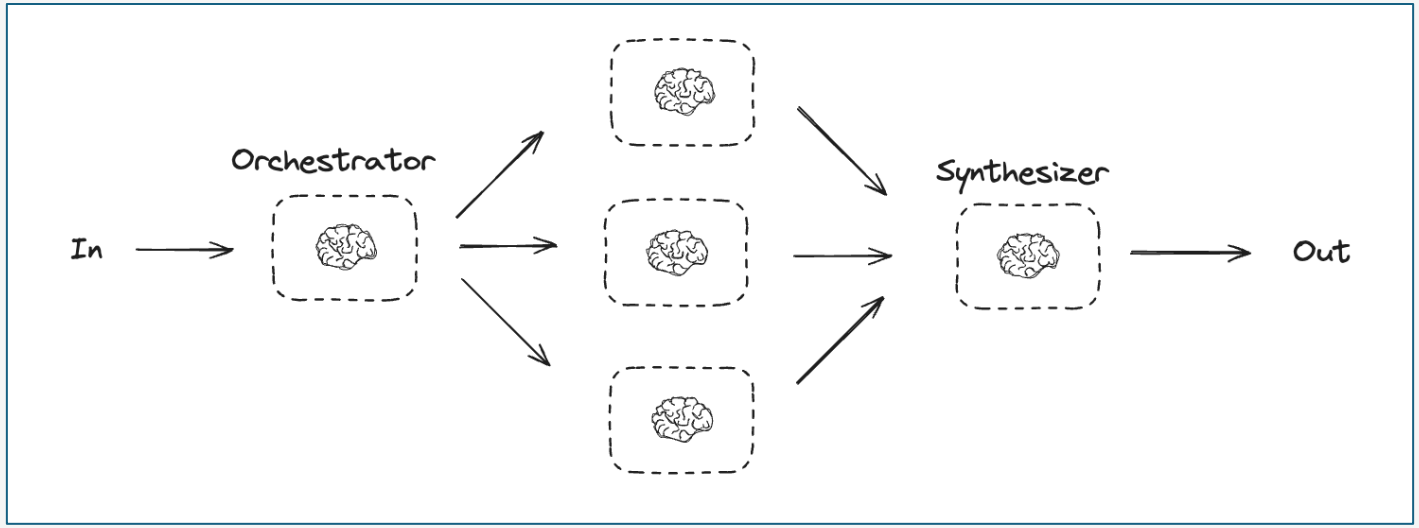
- LLM은 긴 문맥을 입력받을 수 있어도, **중요한 정보가 문맥의 중간에 있으면 처음이나 끝에 있을 때보다 잘 찾지 못한다.**
- https://arxiv.org/pdf/2307.03172



4-4 랭그래프 & 랭스미스

랭그래프 & 랭스미스

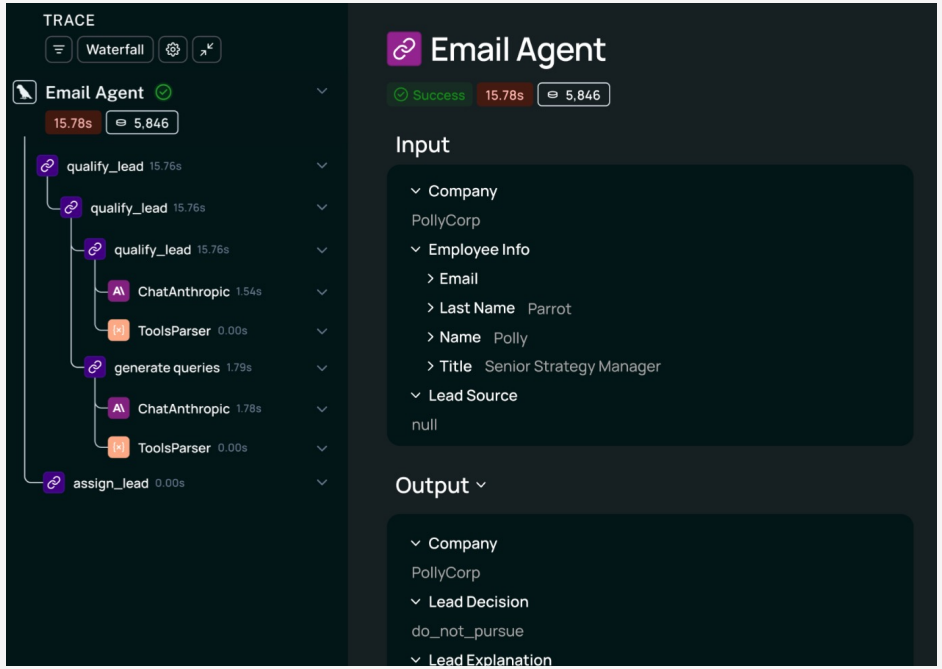
- 랭그래프
  - **오케스트레이션(Orchestration)** 기반 에이전트 → 복잡한 의사결정에 특화
  - 오케스트레이터(지휘자)가 작업을 **분해**, **위임**, **종합**



출처 : https://docs.langchain.com/oss/python/langgraph/workflows-agents

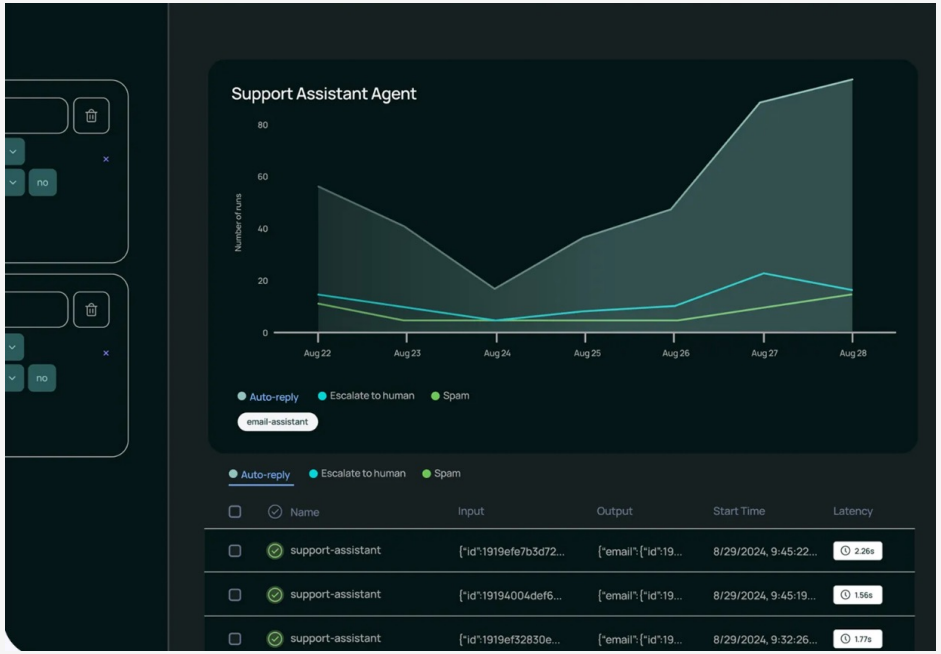
랭그래프 & 랭스미스

- 랭스미스
  - 랭체인, 랭그래프로 만든 애플리케이션 **추적**, **평가**, **모니터링**
  - **추적**
    - LLM 내부의 실행 과정을 시각화
  - **평가**
    - 테스트 데이터를 실행시켜 평가 가능
  - **모니터링**
    - 실제 서비스 배포 후 상태 확인 (Latency, API 비용, 에러)



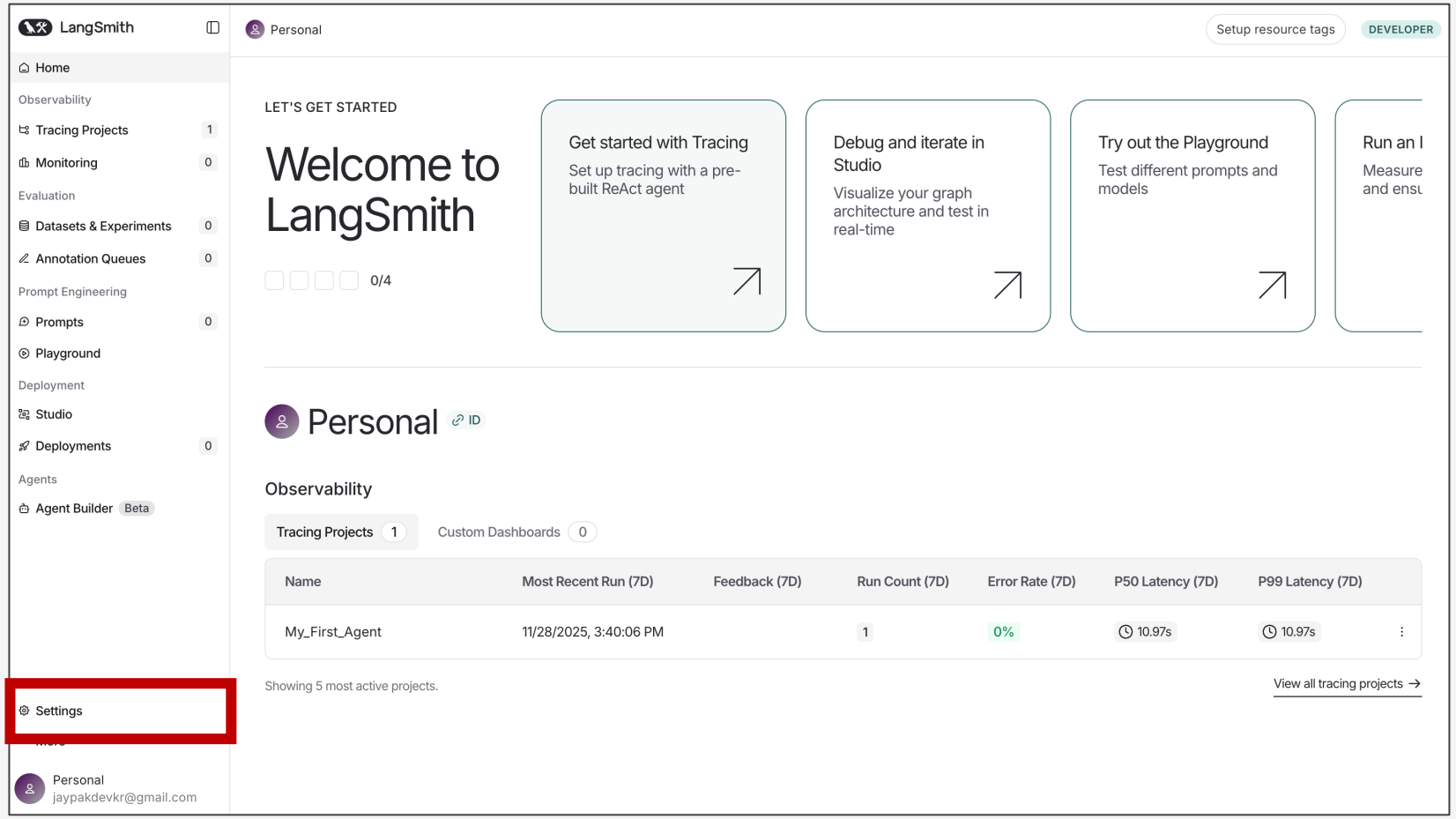
랭그래프 & 랭스미스

- 랭스미스
  - 랭체인, 랭그래프로 만든 애플리케이션 **추적**, **평가**, **모니터링**
  - **추적**
    - LLM 내부의 실행 과정을 시각화
  - **평가**
    - 테스트 데이터를 실행시켜 평가 가능
  - **모니터링**
    - 실제 서비스 배포 후 상태 확인 (Latency, API 비용, 에러)



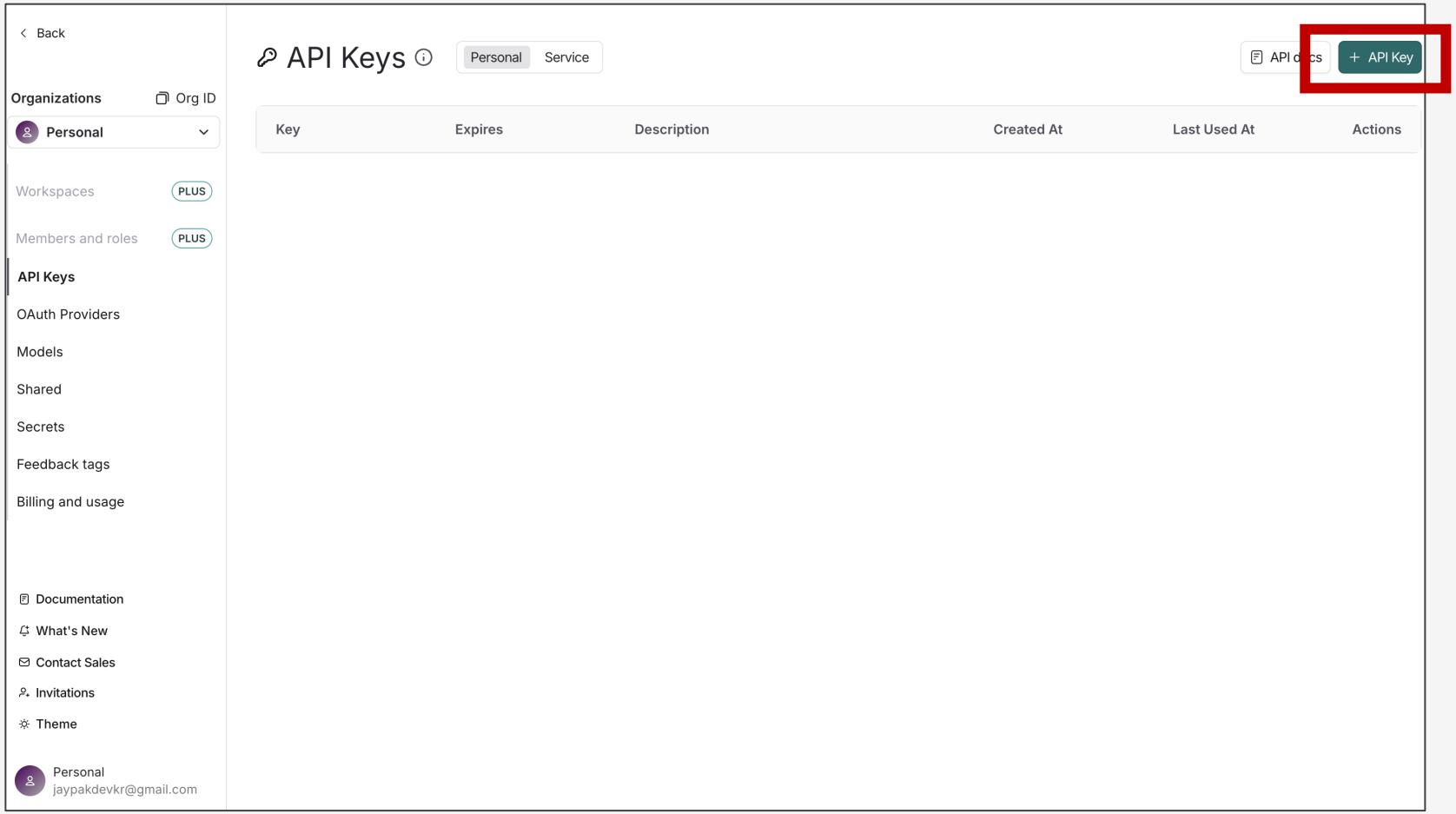
랭그래프 & 랭스미스

- 랭스미스
  - API Key 발급



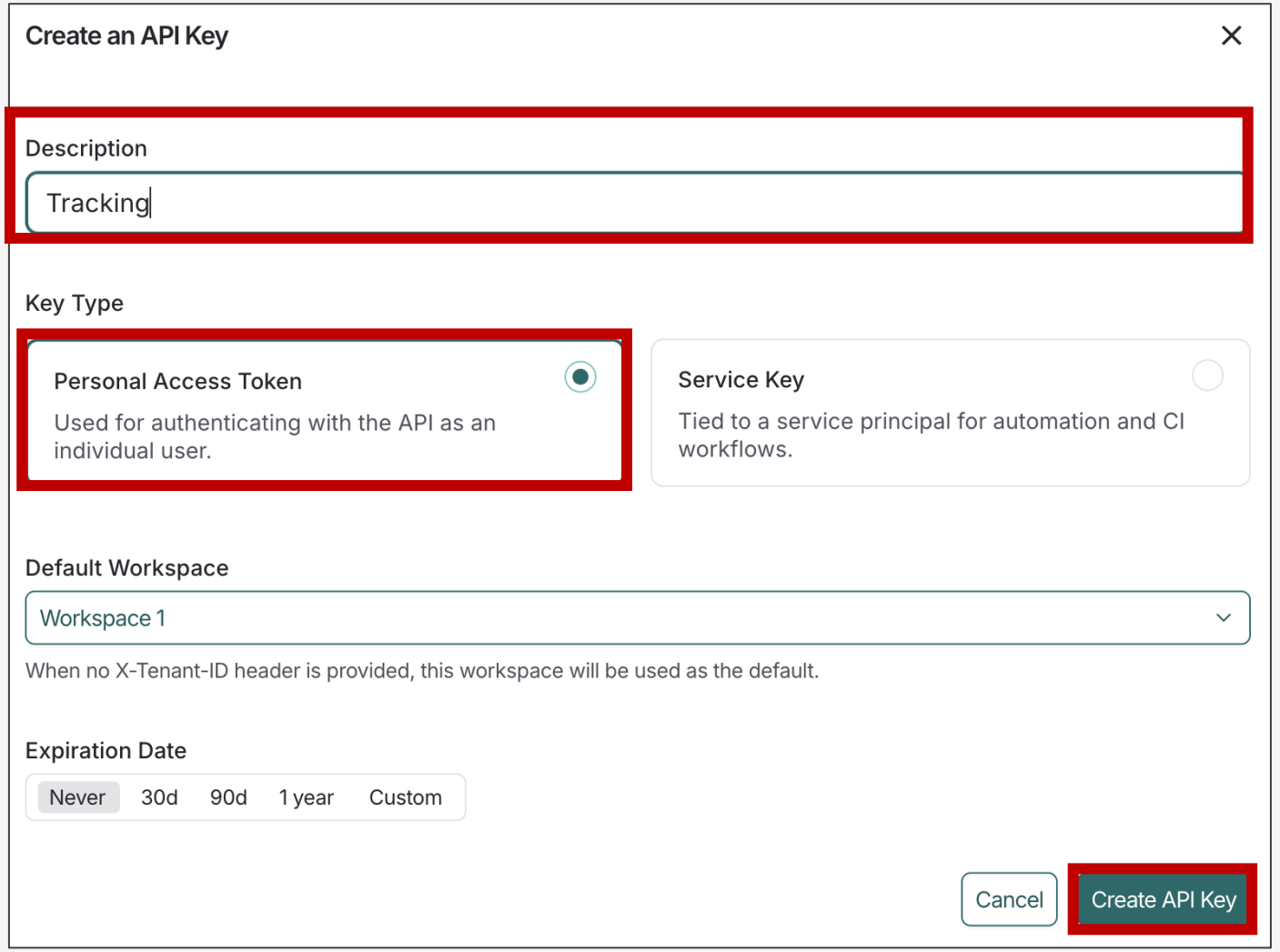
랭그래프 & 랭스미스

- 랭스미스
  - API Key 발급



랭그래프 & 랭스미스

- 랭스미스
  - API Key 발급
    - Description 작성
    - PAT 선택
    - Create API Key 클릭



랭그래프 & 랭스미스

- 랭스미스
  - os 환경변수 설정
    - langchain 라이브러리 초기화
    - langchain, langchain-openai 패키지 재설치
    - os 환경 변수 입력

```
import os

# 1. 랭스미스 추적 기능 켜기
os.environ["LANGCHAIN_TRACING_V2"] = "true"

# 2. 랭스미스 API 키 설정
os.environ["LANGCHAIN_API_KEY"] = "lsv2_pt_9[redacted]"

# 3. 프로젝트 생성
os.environ["LANGCHAIN_PROJECT"] = "My_First_Agent"
```

랭그래프 & 랭스미스

- 랭스미스
  - 랭체인 실행 → 랭스미스 확인

```python
from langchain.chat_models import init_chat_model

model = init_chat_model("gpt-5-nano")
```

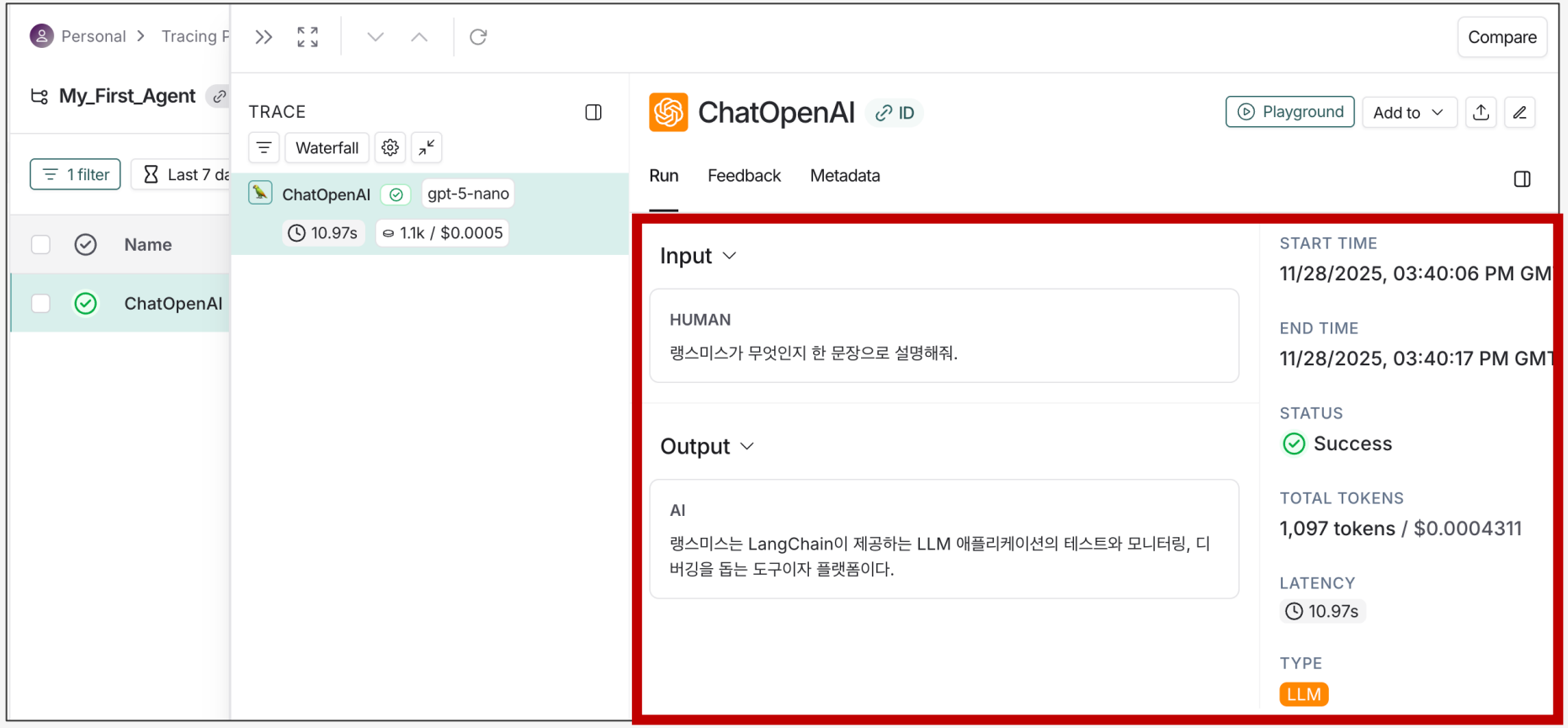
```python
response = model.invoke("랭스미스가 무엇인지 한 문장으로 설명해줘.")
print(f"답변: {response.content}")
```

```
답변: 랭스미스는 LangChain이 제공하는 LLM 애플리케이션의 테스트와 모니터링, 디버깅을 돕는 도구이자 플랫폼이다.
```



랭그래프 & 랭스미스

- 랭스미스
  - 결과 확인



수고하셨습니다.